© 2026 ANSYS, Inc. or affiliated companies
Unauthorized use, distribution, or duplication prohibited.

ansys-aedt-toolkits-antenna

ANSYS, Inc.
Southpointe
2600 Ansys Drive
Canonsburg, PA 15317
ansysinfo@ansys.com
http://www.ansys.com
(T) 724-746-3304
(F) 724-514-9494

Oct. 29, 2026

ANSYS, Inc. and ANSYS Europe, Ltd. are UL registered ISO 9001:2015 companies.

CONTENTS

Useful links: *Installation* | Source repository | Issues

The AEDT Antenna Toolkit is a Python interface for accelerating antenna design using Ansys Electronics Desktop (AEDT). You can launch this toolkit from AEDT or launch it directly from a Python console.

Getting started Learn more about the AEDT Antenna Toolkit and how to install it.

Getting started Antenna wizard Understand how to use the Antenna toolkit wizard.

Antenna wizard API reference Understand the APIs available for the AEDT Antenna Toolkit.

API reference Examples Explore examples that show how to use the API.

Examples Contribute Learn how to contribute to the AEDT Antenna Toolkit codebase or documentation.

Contribute

CHAPTER

ONE

GETTING STARTED

This section explains how to install the AEDT Antenna Toolkit.

Installation Learn how to install the AEDT Antenna Toolkit.

Installation User guide Learn more about the Antenna wizard and how to use it.

User guide CLI guide Learn how to use the command-line interface for antenna design.

CLI guide

1.1 Installation

1.1.1 Download links

The following installers are available for different operating systems:

Table 1: Available Installers

Installer Link	Operating System
Download	Windows
Download	Ubuntu 22.04
Download	Ubuntu 24.04

Visit the Releases page and pull down the latest installer.

1.1.2 Installing the `Antenna Toolkit`

Windows

First step is installing the `Antenna Toolkit`. In order to do so, follow the next steps.

1. Download the necessary installer from the latest available release. The file should be named `Antenna-Toolkit-Installer.exe`.
2. Execute the installer.
3. Search for the `Antenna Toolkit` and run it.

The `Antenna Toolkit` window should appear at this stage.

Linux

Ubuntu

Prerequisites:

1. **OS** supported for **Ubuntu(24.04 and 22.04)**.
2. Update `apt-get` repository and install the following packages with **sudo** privileges: **wget, gnome, libffi-dev, libssl-dev, libsqlite3-dev, libxcb-xinerama0 and build-essential** packages with **sudo** privileges

```
sudo apt-get update -y
sudo apt-get install wget gnome libffi-dev libssl-dev libsqlite3-dev libxcb-
↪xinerama0 build-essential -y
```

3. Install **zlib** package

```
wget https://zlib.net/current/zlib.tar.gz
tar xvzf zlib.tar.gz
cd zlib-*
make clean
./configure
make
sudo make install
```

To install the `Antenna Toolkit`, follow below steps.

1. Download the necessary installer from the latest available release. The file should be named `Antenna-Toolkit-Installer-ubuntu_*.zip`.
2. Execute the below command on the terminal

```
unzip Antenna-Toolkit-Installer-ubuntu_*.zip
./installer.sh
```

3. Search for the `Antenna Toolkit` and run it.

The `Antenna Toolkit` window should appear at this stage.

To uninstall the `Antenna Toolkit`, follow below steps.

1. Go to File menu. Click Uninstall option.
2. Click `Uninstall` button.

1.1.3 Python installation

The Antenna Toolkit can be installed like any other open source package. From PyPI, you can either install both the backend and user interface (UI) methods or install only the backend methods. To install both the backend and UI methods, run this command:

```
pip install ansys-aedt-toolkits-antenna
```

If you only need the common API, install only the backend methods with this command:

```
pip install ansys-aedt-toolkits-antenna
```

1.1.4 For developers

You can be up and running with four lines of code:

```
git clone https://github.com/ansys/pyaedt-toolkits-antenna
cd pyaedt-toolkits-antenna
pip install -e .
```

Now you can run it with:

```
run_toolkit
```

Details

Installing Pytools installer in developer mode allows you to modify the source and enhance it.

Before contributing to the project, please refer to the PyAnsys Developers guide. You need to follow these steps:

1. Start by cloning this repository:

   ```
   git clone https://github.com/ansys/pyaedt-toolkits-antenna
   ```

2. Create a fresh-clean Python environment and activate it. Refer to the official venv documentation if you require further information:

   ```
   # Create a virtual environment
   python -m venv .venv
   # Activate it in a POSIX system
   source .venv/bin/activate
   # Activate it in Windows CMD environment
   .venv\Scripts\activate.bat
   # Activate it in Windows Powershell
   .venv\Scripts\Activate.ps1
   ```

3. Install the project in editable mode:

   ```
   python -m pip install -e . --group tests --group doc
   ```

4. Finally, verify your development installation by running:

   ```
   pytest tests -v
   ```

Style and testing

This project uses pre-commit. Install with:

```
pip install pre-commit
run pre-commit install
```

This now runs `pre-commit` for each commit to ensure you follow project style guidelines. For example:

```
git commit -am 'fix style'
isort....................................................................Passed
black....................................................................Passed
blacken-docs.............................................................Passed
flake8...................................................................Passed
```

(continues on next page)

(continued from previous page)

```
codespell................................................................Passed
pydocstyle...............................................................Passed
check for merge conflicts................................................Passed
debug statements (python)................................................Passed
check yaml...............................................................Passed
trim trailing whitespace.................................................Passed
Validate GitHub Workflows................................................Passed
```

If you need to run it again on all files and not just staged files, run:

```
run pre-commit run --all-files
```

Local build

This app can be deployed as a frozen app using pyinstaller with:

```
pip install -e .[freeze]
run pyinstaller frozen.spec
```

This generates app files at `dist/antenna_toolkit` and you can run it locally by executing `Antenna Toolkit.exe`. For more information on how to create a standalone executable, refer to this section: How to distribute.

Documentation

For building documentation, you can either run the usual rules provided in the Sphinx Makefile:

```
pip install -e . --group doc
doc/make.bat html
# subsequently open the documentation with (under Linux):
<your_browser_name> doc/html/index.html
```

1.2 User guide

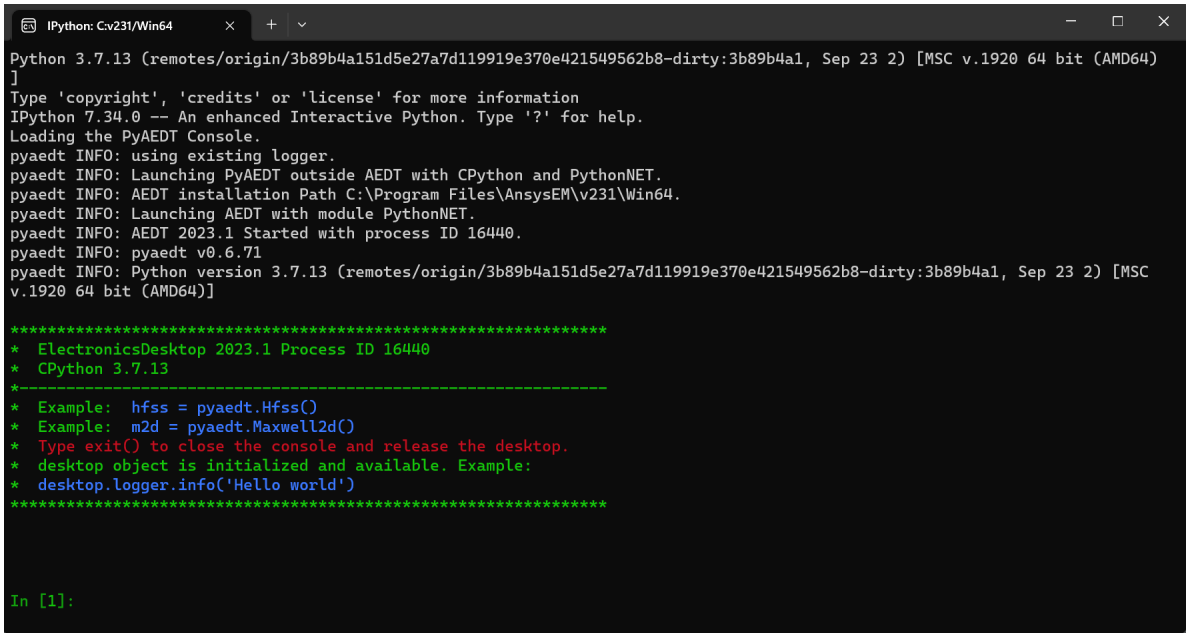
You have multiple options for installing and launching the AEDT Antenna Toolkit:

- You can install the toolkit directly in AEDT using an installation script and then launch it as a wizard. For more information, see *Install toolkit in AEDT and launch the Antenna wizard*.
- You can install the toolkit from a Python console and then launch the Antenna wizard. For more information, see *Install toolkit from Python console and launch the Antenna wizard*.
- You can install the toolkit from a Python console and then use the toolkits APIs. For more information, see *Install toolkit from Python console and use the toolkits APIs*.

1.2.1 Install toolkit in AEDT and launch the Antenna wizard

You can install the AEDT Antenna Toolkit directly in AEDT using the base interpreter from the AEDT installation.

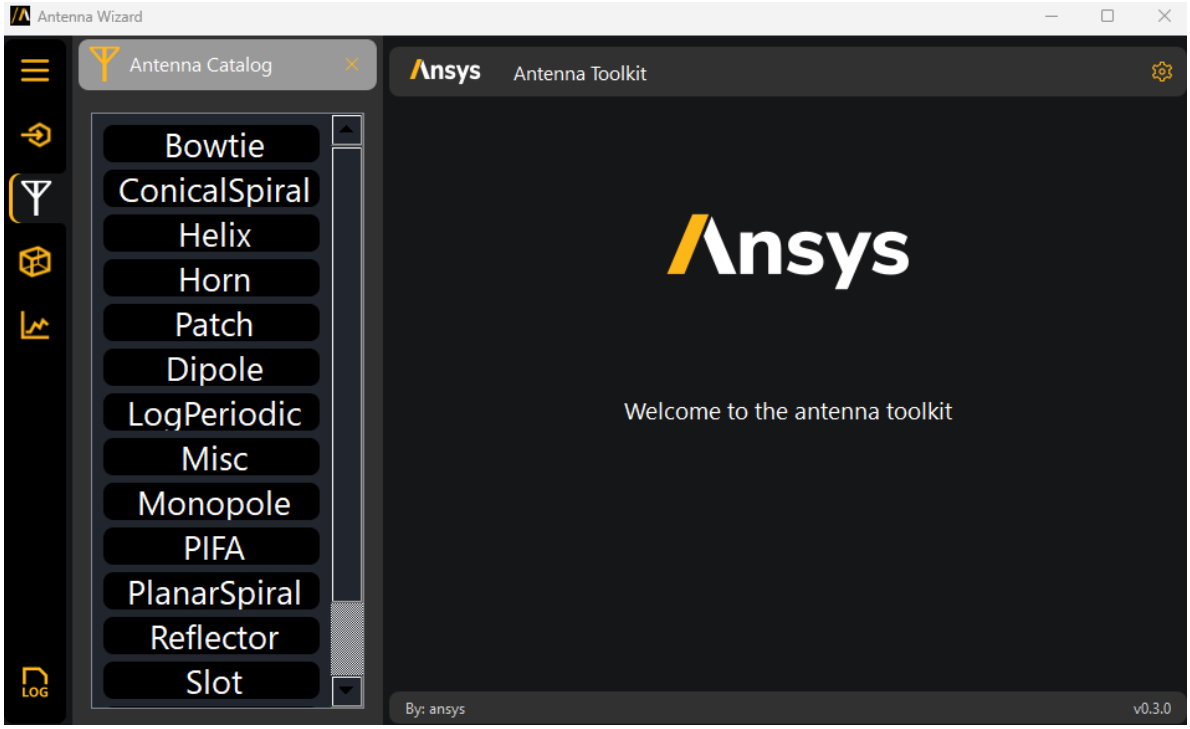
1. From Install from a Python file, follow the steps to install PyAEDT inside AEDT.
2. In AEDT, select **Tools > Toolkit > PyAEDT > Console** to load the PyAEDT console:



3. In the PyAEDT console, run these commands to add the Antenna Toolkit as a wizard (toolkit UI) in AEDT:

```
desktop.add_custom_toolkit("AntennaWizard")
exit()
```

4. In the AEDT toolbar, click the **AntennaWizard** button to open this wizard in AEDT:



The Antenna Toolkit Wizard is connected directly to the AEDT session. For wizard usage information, see *Antenna wizard*.

1.2.2 Install toolkit from Python console and launch the Antenna wizard

You can install the AEDT Antenna Toolkit in a specific Python environment from the AEDT console.

> **Note**
>
> If you have an existing virtual environment, skip step 1.

> **Note**
>
> If you have already installed the toolkit in your virtual environment, skip step 2.

1. Create a fresh-clean Python environment and activate it:

```
# Create a virtual environment
python -m venv .venv

# Activate it in a POSIX system
source .venv/bin/activate

# Activate it in a Windows CMD environment
.venv\Scripts\activate.bat

# Activate it in Windows PowerShell
.venv\Scripts\Activate.ps1
```

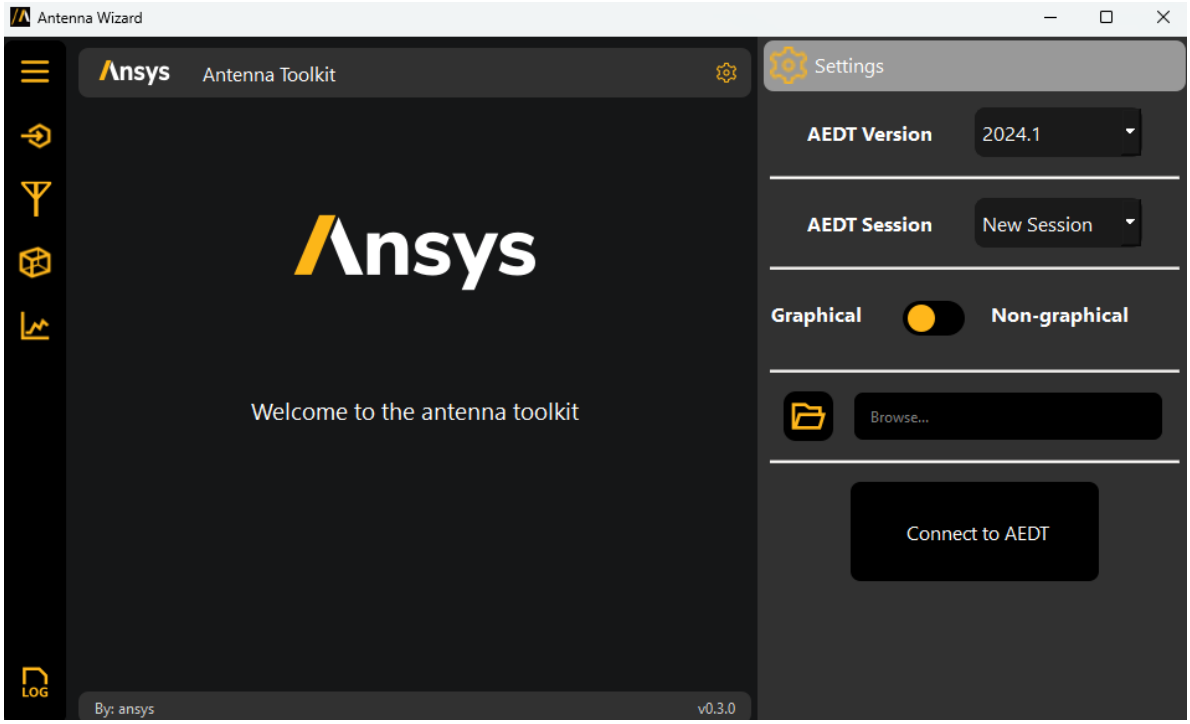
2. Install the toolkit from the GitHub repository:

```
python -m pip install ansys-aedt-toolkits-antenna
```

3. Launch the Antenna Toolkit Wizard:

```
python .venv\Lib\site-packages\ansys\aedt\toolkits\antenna\run_toolkit.py
```

4. On the **AEDT Settings** tab, create an AEDT session or connect to an existing one:



For wizard usage information, see *Antenna wizard*.

1.2.3 Install toolkit from Python console and use the toolkits APIs

You can install the toolkit in a specific Python environment and use the toolkits APIs. The code example included in this topic shows how to use the APIs at the model level and toolkit level.

> **Note**
>
> If you have an existing virtual environment, skip step 1.

> **Note**
>
> If you have already installed the toolkit in your virtual environment, skip step 2.

1. Create a fresh-clean Python environment and activate it:

```
# Create a virtual environment
python -m venv .venv

# Activate it in a POSIX system
source .venv/bin/activate

# Activate it in a Windows CMD environment
.venv\Scripts\activate.bat

# Activate it in Windows PowerShell
.venv\Scripts\Activate.ps1
```

2. Install the toolkit from the GitHub repository:

```
python -m pip install ansys-aedt-toolkits-antenna
```

3. Open a Python console in your virtual environment:

```
python
```

4. From the command line, use the toolkit to create an antenna.

   This code shows how to launch AEDT, create and synthesize a bowtie antenna, and run a simulation in HFSS:

```
# Import required modules
from ansys.aedt.core import Hfss
from ansys.aedt.toolkits.antenna.backend.antenna_models.bowtie import BowTie

# Open AEDT and create an HFSS design
aedtapp = Hfss()

# Create antenna object
oantenna1 = BowTie(aedtapp)

# Change frequency
oantenna1.frequency = 12.0

# Create antenna in HFSS
oantenna1.model_hfss()

# Create setup in HFSS
oantenna1.setup_hfss()

# Release AEDT
aedtapp.release_desktop()
```

5. To create an antenna from the toolkit level, use the `Toolkit` class.

   This code shows how to use the `Toolkit` class to get available antennas and their properties, open AEDT, update antenna properties, and create a bowtie antenna:

```
# Import required modules
import time
from ansys.aedt.toolkits.antenna.backend.api import ToolkitBackend

# Backend object
toolkit = ToolkitBackend()

# Get available antennas
toolkit.available_antennas

# Get properties
properties = toolkit.get_properties()

# Set properties
properties = toolkit.set_properties({"length_unit": "cm"})
```

(continues on next page)

(continued from previous page)

```
# Launch AEDT in a thread
toolkit.launch_aedt()

# Wait until thread is finished
idle = toolkit.wait_to_be_idle()

# Update antenna properties
response = toolkit.set_properties({"substrate_height": 0.1575, "length_unit": "cm"})

# Create a bowtie antenna
toolkit.get_antenna("BowTie")

# Release AEDT
toolkit.release_aedt()
```

1.3 CLI guide

The AEDT Antenna Toolkit provides a powerful command-line interface (CLI) for designing and synthesizing antennas directly from the terminal. The CLI is integrated with PyAEDT and can be accessed using the `pyaedt antenna` command.

1.3.1 Overview

The CLI offers three main commands:

- **list**: Display all available antenna types.
- **synthesize**: Calculate antenna dimensions without connecting to AEDT.
- **create**: Create an antenna in a running AEDT session.

1.3.2 Basic commands

List available antennas

To see all supported antenna types organized by category:

```
pyaedt antenna list
```

This command displays antenna types in a readable format:

```
Available antenna types:

  Bowtie
    bowtie-normal               (BowTieNormal)
    bowtie-rounded              (BowTieRounded)
    bowtie-slot                 (BowTieSlot)
  Patch
    rectangular-patch-edge      (RectangularPatchEdge)
    rectangular-patch-inset     (RectangularPatchInset)
    ...
```

For JSON output (useful for scripts):

```
pyaedt --json antenna list
```

Synthesize antenna

The `synthesize` command calculates antenna dimensions based on your specifications **without requiring AEDT to be running**.

Basic example - Bowtie antenna at 2.4 GHz:

```
pyaedt antenna synthesize bowtie --frequency 2.4
```

Example output:

```
Synthesis results for bowtie (BowTieNormal):

  arm_length                    19.354
  inner_width                   0.968
  outer_width                   17.427
  port_gap                      0.968
  sub_h                         1.6
  sub_x                         77.455
  sub_y                         77.455
```

Patch antenna example:

```
pyaedt antenna synthesize rectangular-patch-edge \
    --frequency 5.8 \
    --substrate-height 1.524
```

Using parameter files (YAML or JSON):

```
pyaedt antenna synthesize bowtie --params-file my-antenna.yaml
```

Example `my-antenna.yaml`:

```yaml
synthesis:
  frequency: 2.4
  substrate_height: 1.6
  permittivity: 4.4
  length_unit: "mm"
```

Create antenna in AEDT

The `create` command creates an antenna in a running AEDT session via gRPC.

Prerequisites:

1. AEDT must be running
2. gRPC server must be enabled (check the port number in AEDT)
3. If no HFSS project is open, the command creates a new project automatically

Basic example:

```
pyaedt antenna create bowtie \
    --port 50051 \
    --frequency 2.4 \
    --substrate-height 1.6
```

With setup creation and analysis options:

```
pyaedt antenna create rectangular-patch-edge \
    --port 50051 \
    --frequency 5.8 \
    --substrate-height 1.524 \
    --create-setup \
    --component-3d \
    --sweep 20 \
    --num-cores 8
```

Specifying project and design:

```
pyaedt antenna create bowtie \
    --port 50051 \
    --project "MyAntennaProject" \
    --design "BowTie_Design" \
    --frequency 2.4
```

1.3.3 Advanced usage

Command-line options

All synthesis parameters from the `Synthesis` Pydantic model are automatically exposed as CLI options. Common options include:

- `--frequency`: Operating frequency in GHz
- `--substrate-height`: Substrate thickness
- `--permittivity`: Relative permittivity (r)
- `--length-unit`: Unit for dimensions (mm, cm, in, etc.)
- `--outer-boundary`: Outer boundary condition

For setup-related options (`create` command only):

- `--create-setup`: Create an analysis setup and sweep (default: False)
- `--component-3d`: Create a 3D component (default: False)
- `--lattice-pair`: Create a lattice pair (default: False)
- `--sweep`: Sweep percentage (default: 20)
- `--num-cores`: Number of cores for analysis (default: 4)

Note

For boolean flags, you can also use `--no-create-setup`, `--no-component-3d`, etc. to explicitly disable them, which is useful when overriding values from parameter files.

To see all available options for a specific command:

```
pyaedt antenna synthesize --help
pyaedt antenna create --help
```

Using parameter files

For complex configurations, use YAML or JSON files:

YAML format:

```yaml
# antenna-config.yaml
synthesis:
  frequency: 2.4
  substrate_height: 1.6
  permittivity: 4.4
  length_unit: "mm"

setup:
  create_setup: true
  component_3d: true
  sweep: 25
  num_cores: 4

param:
  custom_param1: "value1"
  custom_param2: 42
```

JSON format:

```json
{
  "synthesis": {
    "frequency": 2.4,
    "substrate_height": 1.6,
    "permittivity": 4.4
  },
  "setup": {
    "create_setup": true,
    "sweep": 25
  }
}
```

Use with:

```
pyaedt antenna create bowtie --port 50051 --params-file antenna-config.yaml
```

You can override specific parameters from the file using command-line options:

```
# Override setup options from config file
pyaedt antenna create bowtie \
    --port 50051 \
    --params-file antenna-config.yaml \
    --no-create-setup \
    --frequency 3.0
```

Extra parameters

For antenna-specific parameters not in the standard synthesis model, use `--param`:

```
pyaedt antenna synthesize horn-pyramidal \
    --frequency 10 \
    --param "flare_angle=15" \
    --param "waveguide_type=WR90"
```

JSON output mode

All commands support JSON output for integration with scripts and automation. The `--json` flag is used at the PyAEDT level, before the `antenna` command:

```
pyaedt --json antenna synthesize bowtie --frequency 2.4
```

Example JSON output:


```
{
  "data": {
    "antenna": "bowtie",
    "class": "BowTieNormal",
    "parameters": {
      "arm_length": 0.038715,
      "inner_width": 0.001936,
      "outer_width": 0.034854,
      "port_gap": 0.001936,
      "sub_h": 1.6,
      "sub_x": 0.154909,
      "sub_y": 0.154909
    }
  }
}
```

CHAPTER

TWO

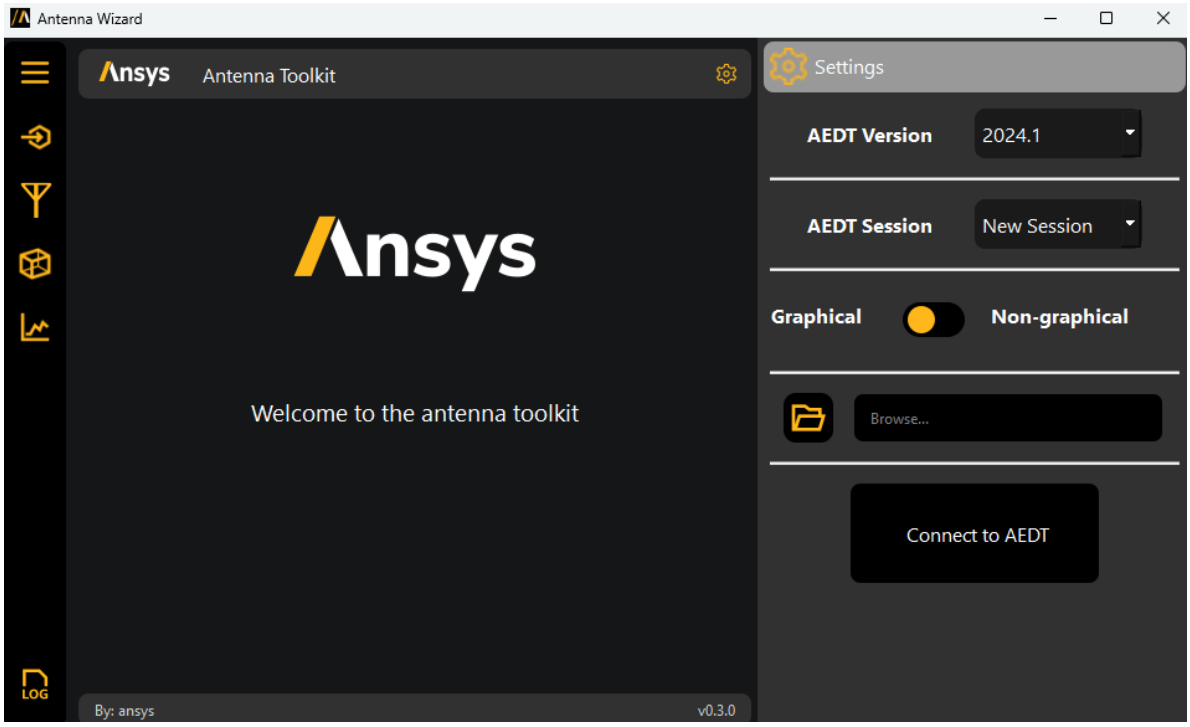
ANTENNA WIZARD

This section describes how to use the Antenna wizard. It assumes that you have already launched the wizard from either the AEDT menu or AEDT console. For toolkit installation and wizard launching information, see these topics:

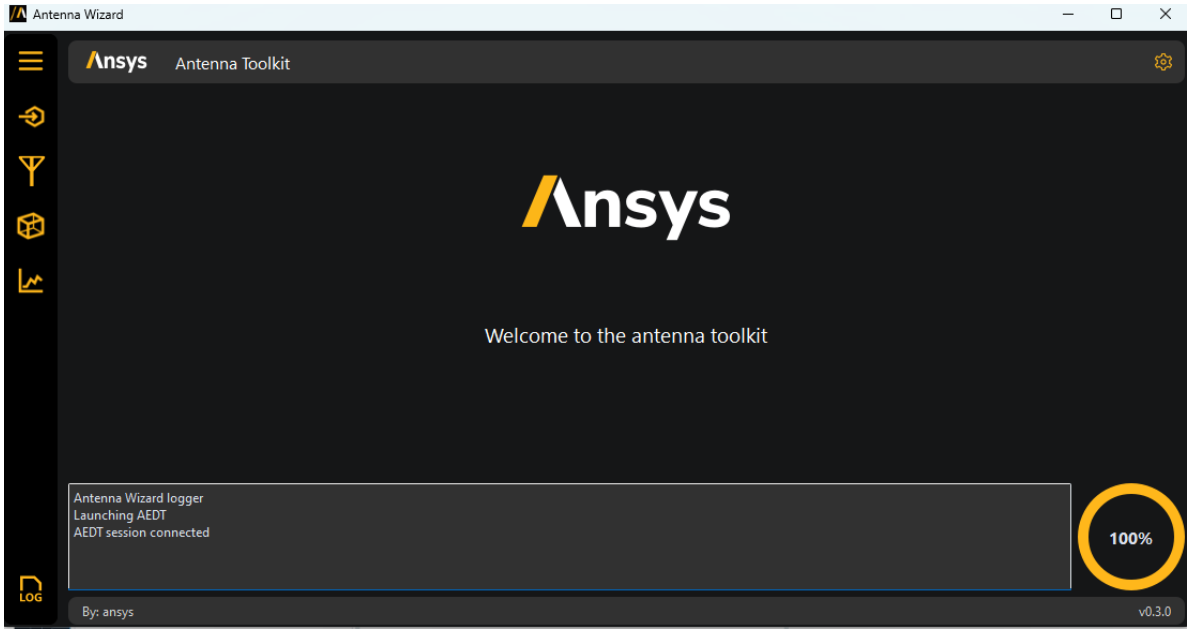
- *Install toolkit in AEDT and launch the Antenna wizard*
- *Install toolkit from Python console and launch the Antenna wizard*

1. On the **Settings** tab, specify settings for either creating an AEDT session or connecting to an existing AEDT session.

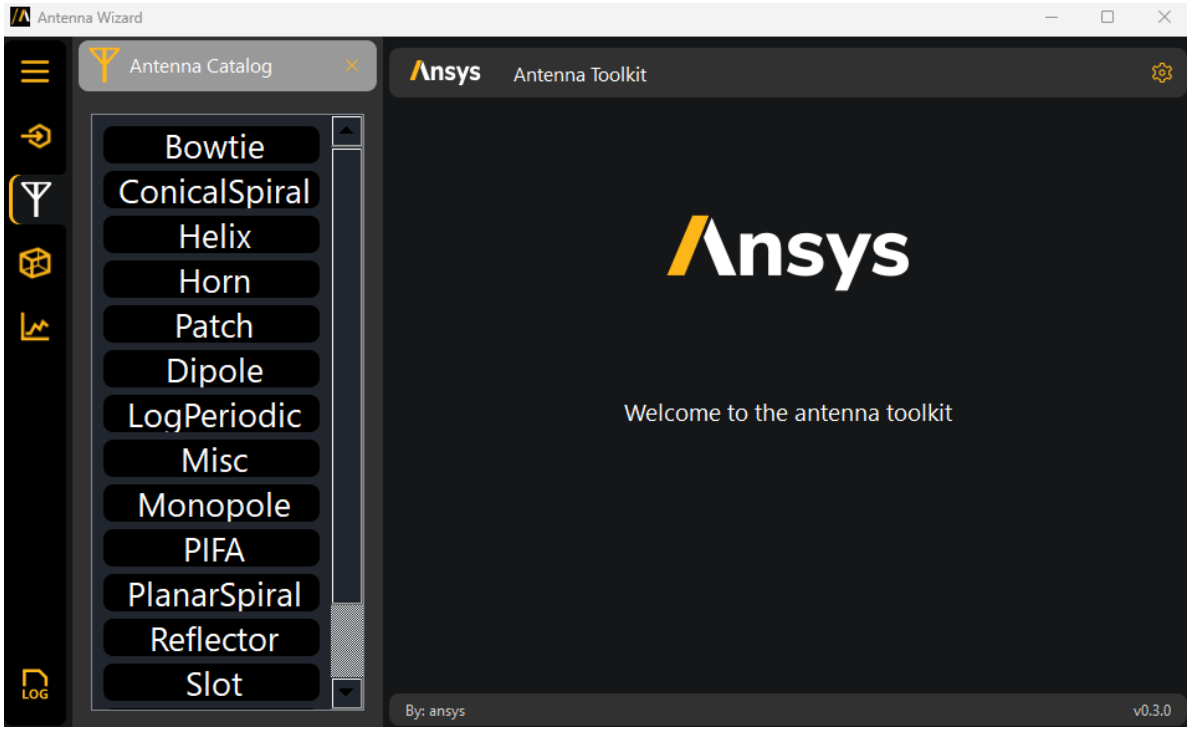
> **Note**
>
> If the Antenna Toolkit Wizard is launched from AEDT, the **Settings** tab does not appear because the toolkit is directly connected to the specific AEDT session.

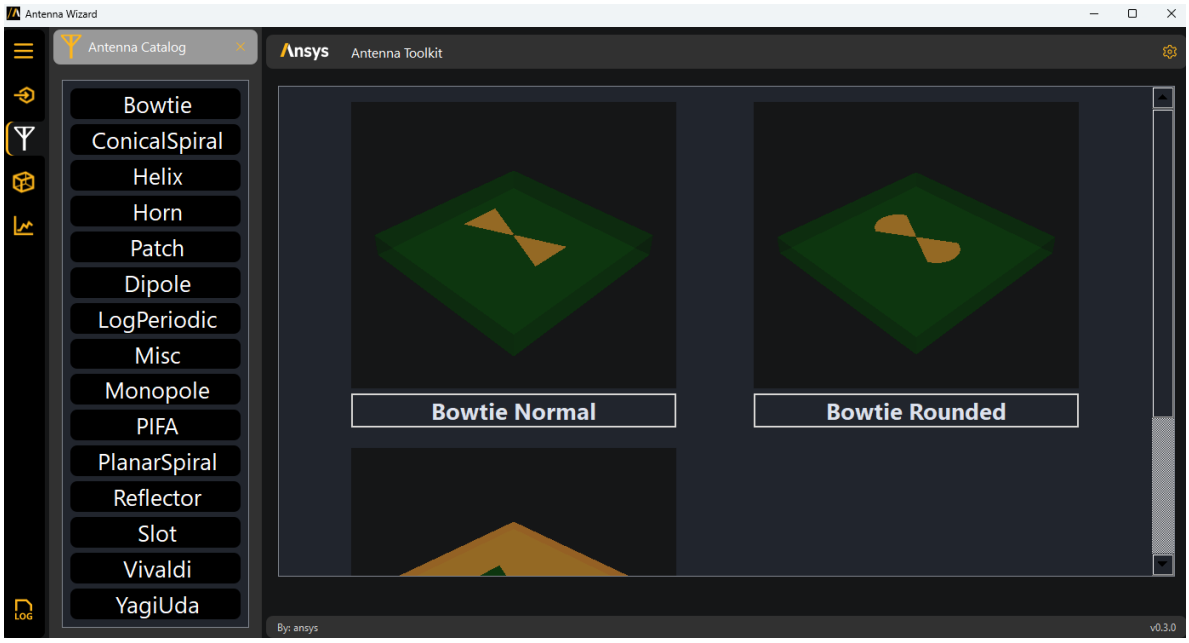


The wizard has a progress circle and a logger box, where you can see the status of every operation.

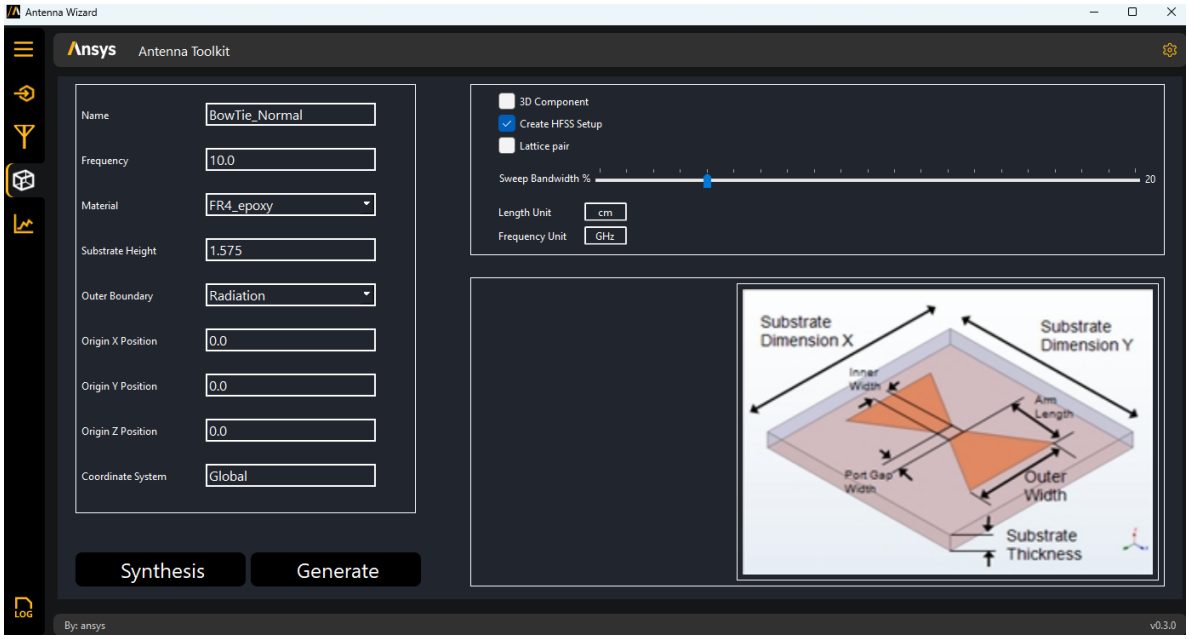


You can choose different antennas from the **Antenna catalog** menu to load the antennas template.



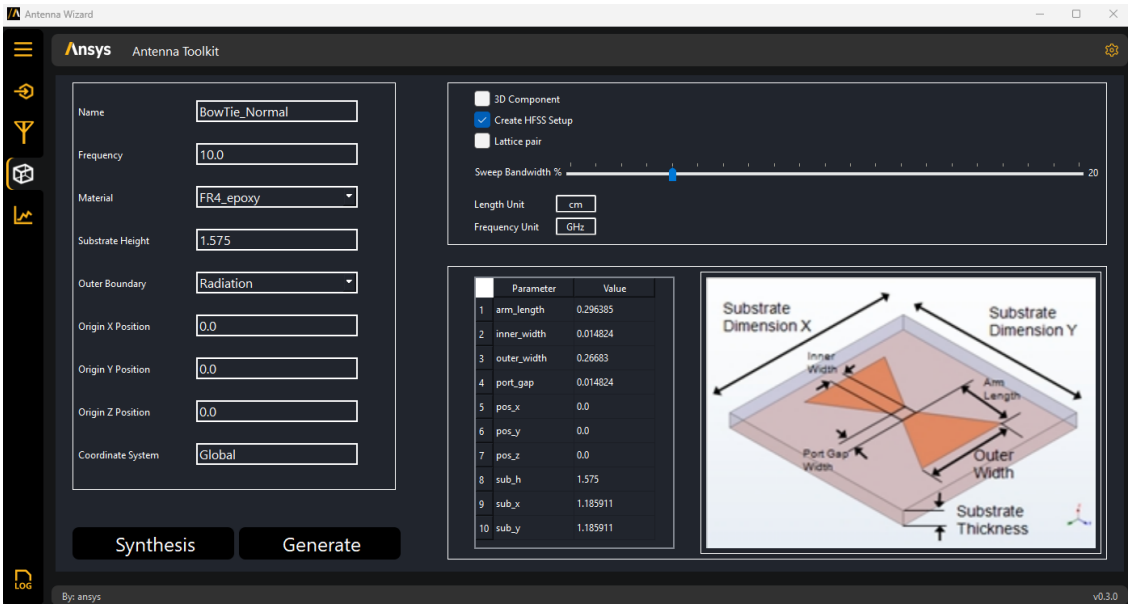


For example, if you select **Antennas > Bowtie > Bowtie Normal**, the central page is updated to the **Synthesis** page and it shows the antenna template:



You have two options: **Synthesis** and **Generate**. The **Generate** button is unavailable if the wizard is not connected to AEDT.

- The **Synthesis** button is for performing the synthesis of the antenna. A connection to AEDT is not needed. You can see the parameters that control the antenna geometry. Additionally, you can do as many syntheses as you want and even change the antenna template.



- The **Generate** button is for creating an HFSS model. It uses the **3D Component**, **Create Hfss Setup**, and **Lattice pair** checkboxes along with the **Sweep Bandwidth %** option It also uses the length and frequency unit to perform the HFSS setup.

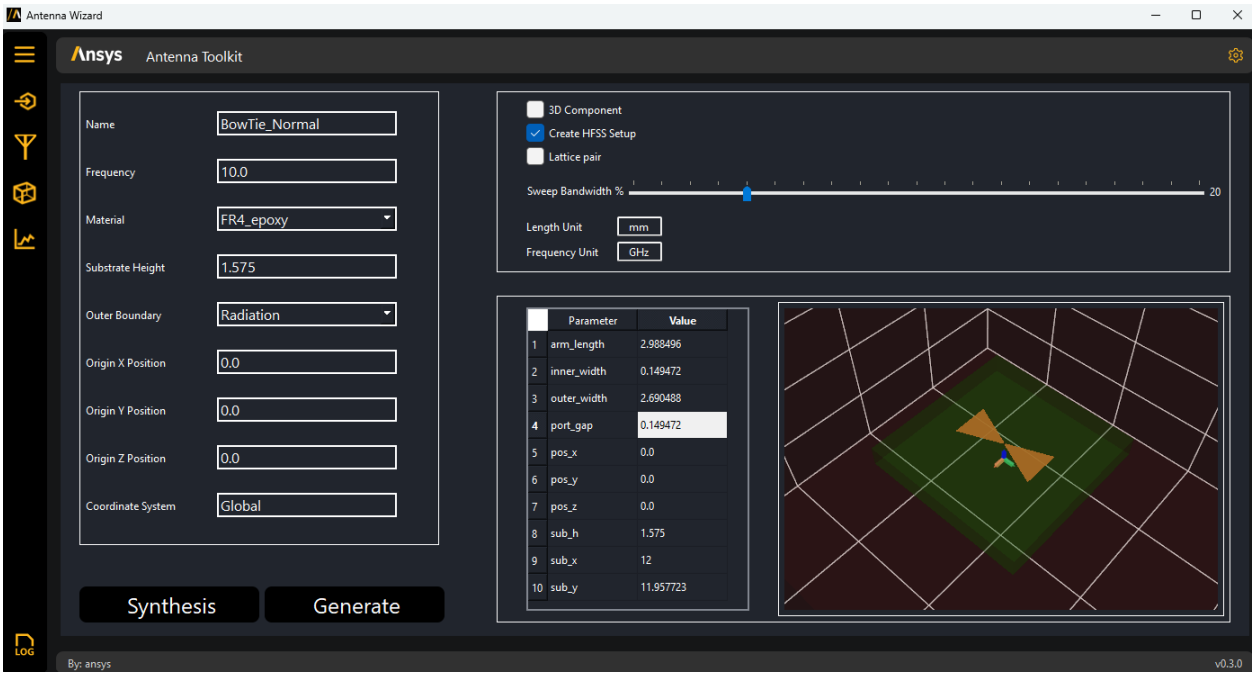
  Descriptions follow for how to use the checkboxes on the **Design** tab:

  – If you select the **3D Component** checkbox, the toolkit creates the antenna and replaces it with a 3D component.

  – If you select the **Generate** checkbox, the toolkit automatically creates the boundaries, excitations, and ports needed to simulate the antenna. Once you create an HFSS model, you cannot create another antenna. Both the **Synthesis** and **Generate** buttons become unavailable. If you want to create another antenna, you must restart the toolkit.

  – If you select the **Lattice pair** checkbox, the toolkit creates a unit cell assigning a lattice pair boundary.

Once you create an antenna, the **Synthesis** tab displays an interactive 3D model rather than the image of the antenna template:



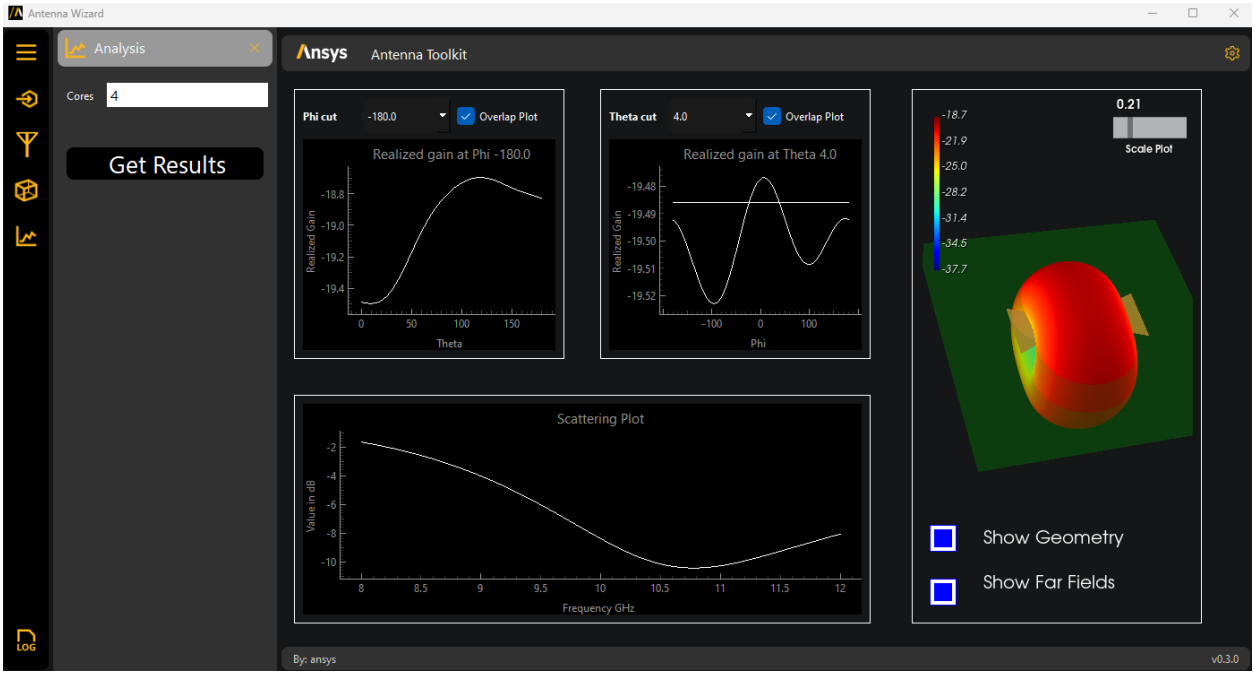
If AEDT is launched in non-graphical mode, you can still see the generated model.

In the wizard, you can modify the parameters interactively, watching both the HFSS model and the interactive 3D plot in the wizard change.

Finally, on the wizards **Analysis** tab, you have the **Get results** button. This second button is unavailable until after you analyze the HFSS design.

When you click **Get results**, the project is analyzed. You can specify the number of cores to use in the simulation.

Once the project is solved, you can click **Get results** on the **Analysis** tab to view results.



CHAPTER

THREE

API REFERENCE

This section provides descriptions of the two APIs available for the AEDT Antenna Toolkit:

- **Toolkit API**: Contains the `Toolkit` class, which provides methods for controlling the toolkit workflow. This API provides methods for synthesizing and creating an antenna. You use the Toolkit API at the toolkit level.
- **Antenna API**: Contains classes for all antenna types available in the toolkit. You use the Antenna API at the model level.

3.1 Toolkit API

The Toolkit API contains the `Toolkit` class, which provides methods for controlling the toolkit workflow. This API provides methods for synthesizing and creating an antenna. You use the Toolkit API at the toolkit level.

The common methods for creating an AEDT session or connecting to an existing AEDT session are provided by the Common PyAEDT toolkit library.

ToolkitBackend()	Provides methods for controlling the toolkit workflow.

3.1.1 ansys.aedt.toolkits.antenna.backend.api.ToolkitBackend

class ansys.aedt.toolkits.antenna.backend.api.**ToolkitBackend**

Provides methods for controlling the toolkit workflow.

This class provides methods for creating an AEDT session, connecting to an existing AEDT session, and synthesizing and creating an antenna in HFSS.

Examples

```
>>> from ansys.aedt.toolkits.antenna.backend.api import ToolkitBackend
>>> import time
>>> toolkit = ToolkitBackend()
>>> msg1 = toolkit.launch_aedt()
>>> toolkit.wait_to_be_idle()
>>> toolkit.get_antenna("BowTie")
```

__init__()

Methods

__init__()	
aedt_sessions()	Get information for the active AEDT sessions.
analyze()	Analyze the design.
connect_aedt()	Connect to an existing AEDT session.
connect_design([app_name])	Connect to an application design.
export_aedt_model([obj_list, export_path, ...])	Export the model in the OBJ format and then encode the file if the encode parameter is enabled.
export_farfield([frequencies, setup, ...])	Export far field data and then encode the file if the encode parameter is enabled.
get_antenna(antenna[, synth_only])	Synthesize and create an antenna in HFSS.
get_design_names()	Get the design names for a specific project.
get_project_name(project_path)	Get the project name from the project path.
get_properties()	Get the toolkit properties.
get_thread_status()	Get the toolkit thread status.
installed_aedt_version()	Get the installed AEDT versions.
is_aedt_connected()	Check if AEDT is connected.
launch_aedt()	Launch AEDT.
launch_thread(process, *args)	Launch the thread.
open_project([project_name])	Open an AEDT project.
release_aedt([close_projects, close_on_exit])	Release AEDT.
save_project([project_path, release_aedt])	Save the project.
scattering_results()	Get antenna scattering results.
serialize_obj_base64(file_path)	Encode a bytes-like object.
set_properties(data)	Assign the passed data to the internal data model.
update_hfss_parameters(key, val)	Update parameters in HFSS.
wait_to_be_idle([timeout])	Wait for the thread to be idle and ready to accept a new task.

You can use the Toolkit API as shown in this example:

```
# Import required modules for the example
import time

# Import backend
from ansys.aedt.toolkits.template.backend.api import ToolkitBackend

# Initialize generic service
toolkit_api = ToolkitBackend()

# Load default properties from a JSON file
properties = toolkit_api.get_properties()

# Set properties
new_properties = {"aedt_version": "2023.1"}
toolkit_api.set_properties(new_properties)
properties = toolkit_api.get_properties()

# Launch AEDT
thread_msg = toolkit_api.launch_thread(toolkit_api.launch_aedt)
```

(continues on next page)

(continued from previous page)

```
# Wait until thread is finished
idle = toolkit_api.wait_to_be_idle()
if not idle:
    print("AEDT not initialized.")
    sys.exit()

# Create geometry
toolkit_api.connect_design("HFSS")

# Create setup when antenna is created
properties.antenna.setup.create_setup = True
properties.antenna.synthesis.outer_boundary = "Radiation"

# Generate antenna
antenna_parameter = toolkit_api.get_antenna("RectangularPatchProbe")

# Release AEDT
toolkit_api.release_aedt()
```

3.2 Antenna API

The Antenna API contains classes for all antenna types available in the toolkit:

3.2.1 Bowtie

This page list the classes available for bowtie antennas:

BowTieNormal(*args, **kwargs)	Manages a bowtie antenna.
BowTieRounded(*args, **kwargs)	Manages a bowtie rounded antenna.
BowTieSlot(*args, **kwargs)	Manages a bowtie slot antenna.

ansys.aedt.toolkits.antenna.backend.antenna_models.bowtie.BowTieNormal

class ansys.aedt.toolkits.antenna.backend.antenna_models.bowtie.**BowTieNormal**(**args*, ***kwargs*)

Manages a bowtie antenna.

This class is accessible through the `Hfss` object [**?**].

Parameters

frequency
[`float`, `optional`] Center frequency. The default is `10.0`.

frequency_unit
[`str`, `optional`] Frequency units. The default is `"GHz"`.

material
[`str`, `optional`] Substrate material. If a material is not defined, a new material, `parametrized`, is defined. The default is `"FR4_epoxy"`.

outer_boundary
[`str`, `optional`] Boundary type to use. The default is `None`. Options are `"FEBI"`, `"PML"`, `"Radiation"`, and `None`.

length_unit
[str, optional] Length units. The default is "mm".

substrate_height
[float, optional] Substrate height. The default is 1.575.

parametrized
[bool, optional] Whether to create a parametrized antenna. The default is True.

Returns

aedt.toolkits.antenna.BowTie
Bowtie antenna object.

Notes

Examples

```
>>> from ansys.aedt.toolkits.antenna.backend.antenna_models.bowtie import␣
↪BowTieNormal
>>> import ansys.aedt.core
>>> app = ansys.aedt.core.Hfss()
>>> oantenna1 = BowTieNormal(app)
>>> oantenna1.frequency = 12.0
>>> oantenna1.model_hfss()
>>> oantenna1.setup_hfss()
>>> oantenna2 = BowTieNormal(app, origin=[200, 50, 0])
>>> oantenna2.model_hfss()
>>> oantenna2.setup_hfss()
>>> app.release_desktop(False, False)
```

__init__(**args*, ***kwargs*)

Methods

__init__(*args, **kwargs)	
create_3dcomponent([component_file, ...])	Create a 3D component of the antenna.
create_lattice_pair([lattice_height, ...])	Create a lattice pair box.
duplicate_along_line(vector[, num_clones])	Duplicate the object along a line.
init_model()	Create a radiation boundary.
model_disco()	Model the bowtie antenna in PyDiscovery.
model_hfss()	Draw a bowtie antenna.
set_variables_in_hfss([not_used])	Create HFSS design variables.
setup_disco()	Set up the model in PyDiscovery.
setup_hfss()	Set up a patch antenna in HFSS.
synthesis()	Antenna synthesis.
update_synthesis_parameters(new_params)	Update the synthesis parameter from the antenna list.

Attributes

antenna_type	
coordinate_system	Reference coordinate system.
element_1_port_phase	

continues on next page

Table 5 – continued from previous page

element_1_rotation_angle	
element_2_port_phase	
element_2_rotation_angle	
element_3_port_phase	
element_3_rotation_angle	
element_4_port_phase	
element_4_rotation_angle	
excitation_type	Excitation type.
feed_rotation_angle	
frequency	Center frequency.
frequency_unit	Frequency units.
length_unit	Length unit.
material	Substrate material.
material_properties	Substrate material properties.
name	Antenna name.
number_of_patches_x	
number_of_patches_y	
origin	Antenna origin.
outer_boundary	Outer boundary.
substrate_height	Substrate height.

ansys.aedt.toolkits.antenna.backend.antenna_models.bowtie.BowTieRounded

class ansys.aedt.toolkits.antenna.backend.antenna_models.bowtie.**BowTieRounded**(**args*, ***kwargs*)

Manages a bowtie rounded antenna.

This class is accessible through the `Hfss` object [**?**].

Parameters

frequency
[`float`, `optional`] Center frequency. The default is `10.0`.

frequency_unit
[`str`, `optional`] Frequency units. The default is `"GHz"`.

material
[`str`, `optional`] Substrate material. If a material is not defined, a new material, `parametrized`, is defined. The default is `"FR4_epoxy"`.

outer_boundary
[`str`, `optional`] Boundary type to use. The default is `None`. Options are `"FEBI"`, `"PML"`, `"Radiation"`, and `None`.

length_unit
[`str`, `optional`] Length units. The default is `"mm"`.

substrate_height
[`float`, `optional`] Substrate height. The default is `1.575`.

parametrized
[bool, `optional`] Whether to create a parametrized antenna. The default is `True`.

Returns

aedt.toolkits.antenna.BowTieRounded
Patch antenna object.

Notes

Examples

```
>>> from ansys.aedt.toolkits.antenna.backend.antenna_models.bowtie import␣
↪BowTieRounded
>>> import ansys.aedt.core
>>> app = ansys.aedt.core.Hfss()
>>> oantenna1 = BowTieRounded(app)
>>> oantenna1.frequency = 12.0
>>> oantenna1.model_hfss()
>>> oantenna1.setup_hfss()
>>> oantenna2 = BowTieRounded(app, origin=[200, 50, 0])
>>> oantenna2.model_hfss()
>>> oantenna2.setup_hfss()
>>> app.release_desktop(False, False)
```

__init__(**args*, ***kwargs*)

Methods

__init__(*args, **kwargs)	
create_3dcomponent([component_file, ...])	Create a 3D component of the antenna.
create_lattice_pair([lattice_height, ...])	Create a lattice pair box.
duplicate_along_line(vector[, num_clones])	Duplicate the object along a line.
init_model()	Create a radiation boundary.
model_disco()	Model in PyDiscovery.
model_hfss()	Draw a bowtie rounded antenna.
set_variables_in_hfss([not_used])	Create HFSS design variables.
setup_disco()	Set up the model in PyDiscovery.
setup_hfss()	Set up a patch antenna in HFSS.
synthesis()	Antenna synthesis.
update_synthesis_parameters(new_params)	Update the synthesis parameter from the antenna list.

Attributes

antenna_type	
coordinate_system	Reference coordinate system.
element_1_port_phase	
element_1_rotation_angle	
element_2_port_phase	
element_2_rotation_angle	
element_3_port_phase	
element_3_rotation_angle	
element_4_port_phase	
element_4_rotation_angle	
excitation_type	Excitation type.
feed_rotation_angle	
frequency	Center frequency.
frequency_unit	Frequency units.
length_unit	Length unit.

continues on next page

Table 7 – continued from previous page

material	Substrate material.
material_properties	Substrate material properties.
name	Antenna name.
number_of_patches_x	
number_of_patches_y	
origin	Antenna origin.
outer_boundary	Outer boundary.
substrate_height	Substrate height.

ansys.aedt.toolkits.antenna.backend.antenna_models.bowtie.BowTieSlot

class ansys.aedt.toolkits.antenna.backend.antenna_models.bowtie.**BowTieSlot**(**args*, ***kwargs*)

Manages a bowtie slot antenna.

This class is accessible through the Hfss object [**?**].

Parameters

frequency
[float, optional] Center frequency. The default is 10.0.

frequency_unit
[str, optional] Frequency units. The default is "GHz".

material
[str, optional] Substrate material. If a material is not defined, a new material, parametrized, is defined. The default is "FR4_epoxy".

outer_boundary
[str, optional] Boundary type to use. The default is None. Options are "FEBI", "PML", "Radiation", and None.

length_unit
[str, optional] Length units. The default is "mm".

substrate_height
[float, optional] Substrate height. The default is 0.1575.

parametrized
[bool, optional] Whether to create a parametrized antenna. The default is True.

Returns

aedt.toolkits.antenna.BowTieSlot
Bowtie antenna object.

Notes

Examples

```
>>> from ansys.aedt.toolkits.antenna.backend.antenna_models.bowtie import BowTieSlot
>>> import ansys.aedt.core
>>> app = ansys.aedt.core.Hfss()
>>> oantenna1 = BowTieSlot(app)
>>> oantenna1.frequency = 12.0
>>> oantenna1.model_hfss()
>>> oantenna1.setup_hfss()
```

(continues on next page)

(continued from previous page)

```
>>> oantenna2 = BowTieSlot(app, origin=[200, 50, 0])
>>> oantenna2.model_hfss()
>>> oantenna2.setup_hfss()
>>> app.release_desktop(False, False)
```

__init__(**args*, ***kwargs*)

Methods

__init__(*args, **kwargs)	
create_3dcomponent([component_file, ...])	Create a 3D component of the antenna.
create_lattice_pair([lattice_height, ...])	Create a lattice pair box.
duplicate_along_line(vector[, num_clones])	Duplicate the object along a line.
init_model()	Create a radiation boundary.
model_disco()	Model in PyDiscovery.
model_hfss()	Draw a bowtie slot antenna.
set_variables_in_hfss([not_used])	Create HFSS design variables.
setup_disco()	Set up the model in PyDiscovery.
setup_hfss()	Set up a patch antenna in HFSS.
synthesis()	Antenna synthesis.
update_synthesis_parameters(new_params)	Update the synthesis parameter from the antenna list.

Attributes

antenna_type	
coordinate_system	Reference coordinate system.
element_1_port_phase	
element_1_rotation_angle	
element_2_port_phase	
element_2_rotation_angle	
element_3_port_phase	
element_3_rotation_angle	
element_4_port_phase	
element_4_rotation_angle	
excitation_type	Excitation type.
feed_rotation_angle	
frequency	Center frequency.
frequency_unit	Frequency units.
length_unit	Length unit.
material	Substrate material.
material_properties	Substrate material properties.
name	Antenna name.
number_of_patches_x	
number_of_patches_y	
origin	Antenna origin.
outer_boundary	Outer boundary.
substrate_height	Substrate height.

You must use these methods from PyAEDT as shown in this example:

```
from ansys.aedt.core import Hfss

from ansys.aedt.toolkits.antenna.backend.antenna_models.bowtie import BowTieNormal

aedtapp = Hfss()

# Create antenna
oantenna1 = BowTieNormal(app)
oantenna1.frequency = 12.0
oantenna1.model_hfss()
oantenna1.setup_hfss()
...
aedtapp.release_desktop()
```

3.2.2 Common

This pages lists common methods available in the Antenna API:

TransmissionLine([frequency, frequency_unit])	Provides base methods common to transmission line calculations.
StandardWaveguide([frequency, frequency_unit])	Provides base methods common to standard waveguides.

ansys.aedt.toolkits.antenna.backend.antenna_models.common.TransmissionLine

class ansys.aedt.toolkits.antenna.backend.antenna_models.common.**TransmissionLine**(*frequency=10*, *frequency_unit='GHz'*)

Provides base methods common to transmission line calculations.

Parameters

frequency
[float, optional] Center frequency. The default is 10.0.

frequency_unit
[str, optional] Frequency units. The default is "GHz".

Returns

aedt.toolkits.antenna.common.TransmissionLine
Transmission line calculator object.

Examples

```
>>> from ansys.aedt.toolkits.antenna.backend.antenna_models.common import␣
↪TransmissionLine
>>> tl_calc = TransmissionLine(frequency=2)
>>> tl_calc.stripline_calculator(substrate_height=10, permittivity=2.2,␣
↪impedance=60)
```

__init__(*frequency=10*, *frequency_unit='GHz'*)

Methods

__init__([frequency, frequency_unit])	
microstrip_calculator(substrate_height, ...)	Use the micro strip line calculator to calculate line width and length.
stripline_calculator(substrate_height, ...)	Use the strip line calculator to calculate line width.
suspended_strip_calculator(wavelength, w1, ...)	Use the suspended strip line calculator to calculate effective permittivity.

ansys.aedt.toolkits.antenna.backend.antenna_models.common.StandardWaveguide

class ansys.aedt.toolkits.antenna.backend.antenna_models.common.**StandardWaveguide**(*frequency=10*, *frequency_unit='GHz'*)

Provides base methods common to standard waveguides.

Parameters

frequency
[float, optional] Center frequency. The default is 10.0.

frequency_unit
[str, optional] Frequency units. The default is "GHz".

Returns

aedt.toolkits.antenna.common.StandardWaveguide
Standard waveguide object.

Examples

```
>>> from ansys.aedt.toolkits.antenna.common import StandardWaveguide
>>> wg_calc = StandardWaveguide()
>>> wg_dim = wg_calc.get_waveguide_dimensions("WR-75")
```

__init__(*frequency=10*, *frequency_unit='GHz'*)

Methods

__init__([frequency, frequency_unit])	
find_waveguide(freq[, units])	Find the closest standard waveguide for the operational frequency.
get_waveguide_dimensions(name[, units])	Get waveguide dimensions.

Attributes

waveguide_list	Standard waveguide list.
wg	

You must use these methods from PyAEDT as shown in this example:

```
from ansys.aedt.toolkits.antenna.backend.antenna_models.common import TransmissionLine

# Transmission line calculation
tl_calc = TransmissionLine(frequency=2)
tl_calc.stripline_calculator(substrate_height=10, permittivity=2.2, impedance=60)
```

3.2.3 Conical spiral

This page lists the classes available for conical spiral antennas:

Archimedean(*args, **kwargs)	Manages conical archimedeal spiral antenna.

ansys.aedt.toolkits.antenna.backend.antenna_models.conical_spiral.Archimedean

class ansys.aedt.toolkits.antenna.backend.antenna_models.conical_spiral.**Archimedean**(**args*, ***kwargs*)

Manages conical archimedeal spiral antenna.

This class is accessible through the app hfss object [**?**].

Parameters

frequency
[float, optional] Center frequency. The default is 10.0.

frequency_unit
[str, optional] Frequency units. The default is "GHz".

material
[str, optional] Horn material. If a material is not defined, a new material, parametrized, is defined. The default is "pec".

outer_boundary
[str, optional] Boundary type to use. The default is None. Options are "FEBI", "PML", "Radiation", and None.

length_unit
[str, optional] Length units. The default is "mm".

parametrized
[bool, optional] Whether to create a parametrized antenna. The default is True.

Returns

aedt.toolkits.antenna.Archimedean
Conical archimedean spiral object.

Notes

Examples

```
>>> from ansys.aedt.core import Hfss
>>> from ansys.aedt.toolkits.antenna.backend.antenna_models.conical_spiral import␣
↪Archimedean
>>> hfss = Hfss()
>>> antenna = Archimedean(
...     hfss,
```

(continues on next page)

(continued from previous page)

```
...     start_frequency=20.0,
...     stop_frequency=50.0,
...     frequency_unit="GHz",
...     outer_boundary="Radiation",
...     length_unit="mm",
...     antenna_name="Archimedean",
...     origin=[1, 100, 50],
... )
>>> antenna.model_hfss()
>>> antenna.setup_hfss()
>>> hfss.release_desktop(False, False)
```

__init__(**args*, ***kwargs*)

Methods

__init__(*args, **kwargs)	
create_3dcomponent([component_file, ...])	Create a 3D component of the antenna.
create_lattice_pair([lattice_height, ...])	Create a lattice pair box.
duplicate_along_line(vector[, num_clones])	Duplicate the object along a line.
init_model()	Create a radiation boundary.
model_disco()	Model in PyDiscovery.
model_hfss()	Draw a conical archimedean spiral antenna.
set_variables_in_hfss([not_used])	Create HFSS design variables.
setup_disco()	Set up in PyDiscovery.
setup_hfss()	Set up an antenna in HFSS.
synthesis()	Antenna synthesis.
update_synthesis_parameters(new_params)	Update the synthesis parameter from the antenna list.

Attributes

antenna_type	
coordinate_system	Reference coordinate system.
excitation_type	Excitation type.
frequency	Central frequency.
frequency_unit	Frequency units.
length_unit	Length unit.
material	Horn material.
name	Antenna name.
origin	Antenna origin.
outer_boundary	Outer boundary.
start_frequency	Start frequency.
stop_frequency	Stop frequency.

You must use these methods from PyAEDT as shown in this example:

```
from ansys.aedt.core import Hfss

from ansys.aedt.toolkits.antenna.backend.antenna_models.conical_spiral import (
    Archimedean,
```

(continues on next page)

(continued from previous page)

```
)

aedtapp = Hfss()

# Create antenna
oantenna1 = RectangularPatchProbe(app)
oantenna1.frequency = 12.0
oantenna1.model_hfss()
oantenna1.setup_hfss()
...
aedtapp.release_desktop()
```

3.2.4 Custom

This page list the classes available for custom antennas:

GPSPatchCeramic(*args, **kwargs)	Manages the ACT-derived ceramic GPS patch antenna.

ansys.aedt.toolkits.antenna.backend.antenna_models.custom.GPSPatchCeramic

class ansys.aedt.toolkits.antenna.backend.antenna_models.custom.**GPSPatchCeramic**(**args*, ***kwargs*)

Manages the ACT-derived ceramic GPS patch antenna.

This antenna is modeled after the ACT (ANSYS Customization Toolkit) reference design for a GPS L1-band (1.575 GHz) ceramic patch antenna with a coaxial feed.

Parameters

***args**
[list] Positional arguments forwarded to CommonPatch.

****kwargs**
[dict] Keyword arguments forwarded to CommonPatch.

Examples

Create a GPS ceramic patch antenna at the default frequency of 1.575 GHz:

```
>>> from ansys.aedt.toolkits.antenna.backend.antenna_models.custom import␣
↪GPSPatchCeramic
>>> antenna = GPSPatchCeramic()
```

__init__(**args*, ***kwargs*)

Initialize the GPS ceramic patch antenna and compute synthesis parameters.

Methods

__init__(*args, **kwargs)	Initialize the GPS ceramic patch antenna and compute synthesis parameters.
create_3dcomponent([component_file, ...])	Create a 3D component of the antenna.
create_lattice_pair([lattice_height, ...])	Create a lattice pair box.

continues on next page

Table 18 – continued from previous page

duplicate_along_line(vector[, num_clones])	Duplicate the object along a line.
init_model()	Create a radiation boundary.
model_disco()	Model the antenna in PyDiscovery.
model_hfss()	Draw the ACT-derived ceramic GPS patch antenna in HFSS.
set_variables_in_hfss([not_used])	Create HFSS design variables.
setup_disco()	Set up the antenna model in PyDiscovery.
setup_hfss()	Set up a patch antenna in HFSS.
synthesis()	Scale the ACT reference geometry from the nominal GPS center frequency.
update_synthesis_parameters(new_params)	Update the synthesis parameter from the antenna list.

Attributes

antenna_type	
coordinate_system	Reference coordinate system.
element_1_port_phase	
element_1_rotation_angle	
element_2_port_phase	
element_2_rotation_angle	
element_3_port_phase	
element_3_rotation_angle	
element_4_port_phase	
element_4_rotation_angle	
excitation_type	Excitation type.
feed_rotation_angle	
frequency	Center frequency.
frequency_unit	Frequency units.
length_unit	Length unit.
material	Substrate material.
material_properties	Substrate material properties.
name	Antenna name.
number_of_patches_x	
number_of_patches_y	
origin	Antenna origin.
outer_boundary	Outer boundary.
substrate_height	Substrate height.

You must use these methods from PyAEDT as shown in this example:

```
from ansys.aedt.core import Hfss

from ansys.aedt.toolkits.antenna.backend.antenna_models.custom import GPSPatchCeramic

aedtapp = Hfss()

# Create antenna
oantenna1 = GPSPatchCeramic(app)
oantenna1.frequency = 12.0
oantenna1.model_hfss()
oantenna1.setup_hfss()
```

(continues on next page)

(continued from previous page)

```
...
aedtapp.release_desktop()
```

3.2.5 Helix

This pages lists the classes available for helix antennas:

AxialMode(*args, **kwargs)	Manages an axial mode helix antenna.
AxialModeTaper(*args, **kwargs)	Manages a tapered axial mode helix antenna.
NormalMode(*args, **kwargs)	Manages a normal mode helix antenna.
QuadrifilarOpen(*args, **kwargs)	Manages an open quadrifilar helix antenna.
QuadrifilarShort(*args, **kwargs)	Manages a shorted quadrifilar helix antenna.

ansys.aedt.toolkits.antenna.backend.antenna_models.helix.AxialMode

class ansys.aedt.toolkits.antenna.backend.antenna_models.helix.**AxialMode**(**args*, ***kwargs*)

Manages an axial mode helix antenna.

This class is accessible through the `Hfss` object [**?**].

Parameters

frequency
[`float`, `optional`] Center frequency. The default is `10.0`.

frequency_unit
[`str`, `optional`] Frequency units. The default is `"GHz"`.

material
[`str`, `optional`] Helix material. If the material is not defined, a new material, `parametrized`, is defined. The default is `"pec"`.

outer_boundary
[`str`, `optional`] Boundary type to use. The default is `None`. Options are `"FEBI"`, `"PML"`, `"Radiation"`, and `None`.

length_unit
[`str`, `optional`] Length units. The default is `"mm"`.

parametrized
[bool, `optional`] Whether to create a parametrized antenna. The default is `True`.

Returns

aedt.toolkits.antenna.AxialMode
Antenna object.

Notes

Examples

```
>>> from ansys.aedt.toolkits.antenna.backend.antenna_models.helix import AxialMode
>>> import ansys.aedt.core
>>> app = ansys.aedt.core.Hfss()
>>> oantenna1 = AxialMode(app)
>>> oantenna1.frequency = 12.0
```

(continues on next page)

(continued from previous page)

```
>>> oantenna1.model_hfss()
>>> oantenna1.setup_hfss()
>>> oantenna2 = AxialMode(app, origin=[200, 50, 0])
>>> oantenna2.model_hfss()
>>> oantenna2.setup_hfss()
>>> app.release_desktop(False, False)
```

__init__(**args*, ***kwargs*)

Methods

__init__(*args, **kwargs)	
create_3dcomponent([component_file, ...])	Create a 3D component of the antenna.
create_lattice_pair([lattice_height, ...])	Create a lattice pair box.
duplicate_along_line(vector[, num_clones])	Duplicate the object along a line.
init_model()	Create a radiation boundary.
model_disco()	Model in PyDiscovery.
model_hfss()	Draw an axial mode antenna.
set_variables_in_hfss([not_used])	Create HFSS design variables.
setup_disco()	Set up model in PyDiscovery.
setup_hfss()	Set up an antenna in HFSS.
synthesis()	Antenna synthesis.
update_synthesis_parameters(new_params)	Update the synthesis parameter from the antenna list.

Attributes

antenna_type	
coordinate_system	Reference coordinate system.
direction	Helix direction.
excitation_type	Excitation type.
feeder_length	Helix feeder length.
frequency	Center frequency.
frequency_unit	Frequency units.
gain	Helix expected gain.
length_unit	Length unit.
material	Helix material.
name	Antenna name.
origin	Antenna origin.
outer_boundary	Outer boundary.

ansys.aedt.toolkits.antenna.backend.antenna_models.helix.AxialModeTaper

class ansys.aedt.toolkits.antenna.backend.antenna_models.helix.**AxialModeTaper**(**args*, ***kwargs*)

Manages a tapered axial mode helix antenna.

Notes

__init__(**args*, ***kwargs*)

Methods

__init__(*args, **kwargs)	
create_3dcomponent([component_file, ...])	Create a 3D component of the antenna.
create_lattice_pair([lattice_height, ...])	Create a lattice pair box.
duplicate_along_line(vector[, num_clones])	Duplicate the object along a line.
init_model()	Create a radiation boundary.
model_disco()	Model in PyDiscovery.
model_hfss()	
set_variables_in_hfss([not_used])	Create HFSS design variables.
setup_disco()	Set up model in PyDiscovery.
setup_hfss()	Set up an antenna in HFSS.
synthesis()	
update_synthesis_parameters(new_params)	Update the synthesis parameter from the antenna list.

Attributes

antenna_type	
coordinate_system	Reference coordinate system.
direction	Helix direction.
excitation_type	Excitation type.
feeder_length	Helix feeder length.
frequency	Center frequency.
frequency_unit	Frequency units.
gain	Helix expected gain.
length_unit	Length unit.
material	Helix material.
name	Antenna name.
origin	Antenna origin.
outer_boundary	Outer boundary.

ansys.aedt.toolkits.antenna.backend.antenna_models.helix.NormalMode

class ansys.aedt.toolkits.antenna.backend.antenna_models.helix.**NormalMode**(**args*, ***kwargs*)

Manages a normal mode helix antenna.

Notes

__init__(**args*, ***kwargs*)

Methods

__init__(*args, **kwargs)	
create_3dcomponent([component_file, ...])	Create a 3D component of the antenna.
create_lattice_pair([lattice_height, ...])	Create a lattice pair box.
duplicate_along_line(vector[, num_clones])	Duplicate the object along a line.

continues on next page

Table 25 – continued from previous page

init_model()	Create a radiation boundary.
model_disco()	Model in PyDiscovery.
model_hfss()	
set_variables_in_hfss([not_used])	Create HFSS design variables.
setup_disco()	Set up model in PyDiscovery.
setup_hfss()	Set up an antenna in HFSS.
synthesis()	
update_synthesis_parameters(new_params)	Update the synthesis parameter from the antenna list.

Attributes

antenna_type	
coordinate_system	Reference coordinate system.
direction	Helix direction.
excitation_type	Excitation type.
feeder_length	Helix feeder length.
frequency	Center frequency.
frequency_unit	Frequency units.
gain	Helix expected gain.
length_unit	Length unit.
material	Helix material.
name	Antenna name.
origin	Antenna origin.
outer_boundary	Outer boundary.

ansys.aedt.toolkits.antenna.backend.antenna_models.helix.QuadrifilarOpen

class ansys.aedt.toolkits.antenna.backend.antenna_models.helix.**QuadrifilarOpen**(**args*, ***kwargs*)

Manages an open quadrifilar helix antenna.

Notes

__init__(**args*, ***kwargs*)

Methods

__init__(*args, **kwargs)	
create_3dcomponent([component_file, ...])	Create a 3D component of the antenna.
create_lattice_pair([lattice_height, ...])	Create a lattice pair box.
duplicate_along_line(vector[, num_clones])	Duplicate the object along a line.
init_model()	Create a radiation boundary.
model_disco()	Model in PyDiscovery.
model_hfss()	
set_variables_in_hfss([not_used])	Create HFSS design variables.
setup_disco()	Set up model in PyDiscovery.
setup_hfss()	Set up an antenna in HFSS.
synthesis()	
update_synthesis_parameters(new_params)	Update the synthesis parameter from the antenna list.

Attributes

antenna_type	
coordinate_system	Reference coordinate system.
direction	Helix direction.
excitation_type	Excitation type.
feeder_length	Helix feeder length.
frequency	Center frequency.
frequency_unit	Frequency units.
gain	Helix expected gain.
length_unit	Length unit.
material	Helix material.
name	Antenna name.
origin	Antenna origin.
outer_boundary	Outer boundary.

ansys.aedt.toolkits.antenna.backend.antenna_models.helix.QuadrifilarShort

class ansys.aedt.toolkits.antenna.backend.antenna_models.helix.**QuadrifilarShort**(**args*, ***kwargs*)

Manages a shorted quadrifilar helix antenna.

Notes

__init__(**args*, ***kwargs*)

Methods

__init__(*args, **kwargs)	
create_3dcomponent([component_file, ...])	Create a 3D component of the antenna.
create_lattice_pair([lattice_height, ...])	Create a lattice pair box.
duplicate_along_line(vector[, num_clones])	Duplicate the object along a line.
init_model()	Create a radiation boundary.
model_disco()	Model in PyDiscovery.
model_hfss()	
set_variables_in_hfss([not_used])	Create HFSS design variables.
setup_disco()	Set up model in PyDiscovery.
setup_hfss()	Set up an antenna in HFSS.
synthesis()	
update_synthesis_parameters(new_params)	Update the synthesis parameter from the antenna list.

Attributes

antenna_type	
coordinate_system	Reference coordinate system.
direction	Helix direction.
excitation_type	Excitation type.
feeder_length	Helix feeder length.
frequency	Center frequency.
frequency_unit	Frequency units.

continues on next page

Table 30 – continued from previous page

gain	Helix expected gain.
length_unit	Length unit.
material	Helix material.
name	Antenna name.
origin	Antenna origin.
outer_boundary	Outer boundary.

You must use these methods from PyAEDT as shown in this example:

```
from ansys.aedt.core import Hfss

from ansys.aedt.toolkits.antenna.backend.antenna_models.helix import AxialMode

aedtapp = Hfss()

# Create antenna
oantenna1 = AxialMode(app)
oantenna1.model_hfss()
oantenna1.setup_hfss()
...
aedtapp.release_desktop()
```

3.2.6 Horn

This page lists the classes available for horns:

Conical(*args, **kwargs)	Manages a conical horn antenna.
Corrugated(*args, **kwargs)	Manages a corrugated horn antenna.
Elliptical(*args, **kwargs)	Manages an elliptical horn antenna.
EPlane(*args, **kwargs)	Manages an E plane horn antenna.
HPlane(*args, **kwargs)	Manages an H plane horn antenna.
Pyramidal(*args, **kwargs)	Manages a pyramidal horn antenna.
PyramidalRidged(*args, **kwargs)	Manages a pyramidal ridged horn antenna.
QuadRidged(*args, **kwargs)	Manages a quad-ridged horn antenna.

ansys.aedt.toolkits.antenna.backend.antenna_models.horn.Conical

class ansys.aedt.toolkits.antenna.backend.antenna_models.horn.**Conical**(**args*, ***kwargs*)

Manages a conical horn antenna.

This class is accessible through the app hfss object [**?**].

Parameters

frequency
[float, optional] Center frequency. The default is 10.0.

frequency_unit
[str, optional] Frequency units. The default is "GHz".

material
[str, optional] Horn material. If a material is not defined, a new material, parametrized, is defined. The default is "pec".

outer_boundary
[str, optional] Boundary type to use. The default is None. Options are "FEBI", "PML", "Radiation", and None.

length_unit
[str, optional] Length units. The default is "mm".

parametrized
[bool, optional] Whether to create a parametrized antenna. The default is True.

Returns

aedt.toolkits.antenna.ConicalHorn
Conical horn object.

Notes

Examples

```
>>> from ansys.aedt.toolkits.antenna.backend.antenna_models.horn import Conical
>>> import ansys.aedt.core
>>> oantenna1 = Conical(None)
>>> oantenna1.frequency = 12.0
>>> app = ansys.aedt.core.Hfss()
>>> oantenna1 = Conical(app)
>>> oantenna1.model_hfss()
>>> oantenna1.setup_hfss()
>>> oantenna2 = Conical(app, origin=[0.2, 0.5, 0])
>>> oantenna2.model_hfss()
>>> oantenna2.setup_hfss()
>>> app.release_desktop(False, False)
```

__init__(**args*, ***kwargs*)

Methods

__init__(*args, **kwargs)	
create_3dcomponent([component_file, ...])	Create a 3D component of the antenna.
create_lattice_pair([lattice_height, ...])	Create a lattice pair box.
duplicate_along_line(vector[, num_clones])	Duplicate the object along a line.
init_model()	Create a radiation boundary.
model_disco()	Model in PyDiscovery.
model_hfss()	Draw a conical horn antenna.
set_variables_in_hfss([not_used])	Create HFSS design variables.
setup_disco()	Set up in PyDiscovery.
setup_hfss()	Set up an antenna in HFSS.
synthesis()	Antenna synthesis.
update_synthesis_parameters(new_params)	Update the synthesis parameter from the antenna list.

Attributes

antenna_type	
coordinate_system	Reference coordinate system.

continues on next page

Table 33 – continued from previous page

excitation_type	Excitation type.
frequency	Center frequency.
frequency_unit	Frequency units.
length_unit	Length unit.
material	Horn material.
material_properties	Substrate material properties.
name	Antenna name.
origin	Antenna origin.
outer_boundary	Outer boundary.

ansys.aedt.toolkits.antenna.backend.antenna_models.horn.Corrugated

class ansys.aedt.toolkits.antenna.backend.antenna_models.horn.**Corrugated**(**args*, ***kwargs*)

Manages a corrugated horn antenna.

This class is accessible through the app hfss object [**?**].

Parameters

frequency
[float, optional] Center frequency. The default is 10.0.

frequency_unit
[str, optional] Frequency units. The default is "GHz".

material
[str, optional] Horn material. If a material is not defined, a new material, parametrized, is defined. The default is "pec".

outer_boundary
[str, optional] Boundary type to use. The default is None. Options are "FEBI", "PML", "Radiation", and None.

length_unit
[str, optional] Length units. The default is "mm".

parametrized
[bool, optional] Whether to create a parametrized antenna. The default is True.

Returns

aedt.toolkits.antenna.CorrugatedHorn
Corrugated horn object.

Notes

Examples

```
>>> from ansys.aedt.toolkits.antenna.backend.antenna_models.horn import Corrugated
>>> import ansys.aedt.core
>>> app = ansys.aedt.core.Hfss()
>>> oantenna1 = Corrugated(app)
>>> oantenna1.frequency = 12.0
>>> oantenna1.model_hfss()
>>> oantenna1.setup_hfss()
>>> app.release_desktop(False, False)
```

__init__(**args*, ***kwargs*)

Methods

`__init__`(*args, **kwargs)	
`create_3dcomponent`([component_file, ...])	Create a 3D component of the antenna.
`create_lattice_pair`([lattice_height, ...])	Create a lattice pair box.
`duplicate_along_line`(vector[, num_clones])	Duplicate the object along a line.
`init_model`()	Create a radiation boundary.
`model_disco`()	Model in PyDiscovery.
`model_hfss`()	Draw a conical horn antenna.
`set_variables_in_hfss`([not_used])	Create HFSS design variables.
`setup_disco`()	Set up in PyDiscovery.
`setup_hfss`()	Set up an antenna in HFSS.
`synthesis`()	Antenna synthesis.
`update_synthesis_parameters`(new_params)	Update the synthesis parameter from the antenna list.

Attributes

`antenna_type`	
`coordinate_system`	Reference coordinate system.
`excitation_type`	Excitation type.
`frequency`	Center frequency.
`frequency_unit`	Frequency units.
`length_unit`	Length unit.
`material`	Horn material.
`material_properties`	Substrate material properties.
`name`	Antenna name.
`origin`	Antenna origin.
`outer_boundary`	Outer boundary.

ansys.aedt.toolkits.antenna.backend.antenna_models.horn.Elliptical

class ansys.aedt.toolkits.antenna.backend.antenna_models.horn.**Elliptical**(**args*, ***kwargs*)

Manages an elliptical horn antenna.

This class is accessible through the app hfss object [**?**].

Parameters

frequency
[`float`, `optional`] Center frequency. The default is `10.0`.

frequency_unit
[`str`, `optional`] Frequency units. The default is `"GHz"`.

material
[`str`, `optional`] Horn material. If a material is not defined, a new material, `parametrized`, is defined. The default is `"pec"`.

outer_boundary
[`str`, `optional`] Boundary type to use. The default is `None`. Options are `"FEBI"`, `"PML"`, `"Radiation"`, and `None`.

length_unit
[`str`, `optional`] Length units. The default is `"mm"`.

parametrized
[bool, optional] Whether to create a parametrized antenna. The default is True.

Returns

aedt.toolkits.antenna.EllipticalHorn
Elliptical horn object.

Notes

Examples

```
>>> from ansys.aedt.toolkits.antenna.backend.antenna_models.horn import␣
↪PyramidalRidged
>>> import ansys.aedt.core
>>> app = ansys.aedt.core.Hfss()
>>> oantenna1 = Elliptical(app)
>>> oantenna1.frequency = 12.0
>>> oantenna1.model_hfss()
>>> oantenna1.setup_hfss()
>>> oantenna2 = Elliptical(app, origin=[0.2, 0.5, 0])
>>> oantenna2.model_hfss()
>>> oantenna2.setup_hfss()
>>> app.release_desktop(False, False)
```

__init__(**args*, ***kwargs*)

Methods

__init__(*args, **kwargs)	
create_3dcomponent([component_file, ...])	Create a 3D component of the antenna.
create_lattice_pair([lattice_height, ...])	Create a lattice pair box.
duplicate_along_line(vector[, num_clones])	Duplicate the object along a line.
init_model()	Create a radiation boundary.
model_disco()	Model in PyDiscovery.
model_hfss()	Draw elliptical horn antenna.
set_variables_in_hfss([not_used])	Create HFSS design variables.
setup_disco()	Set up in PyDiscovery.
setup_hfss()	Set up an antenna in HFSS.
synthesis()	Antenna synthesis.
update_synthesis_parameters(new_params)	Update the synthesis parameter from the antenna list.

Attributes

antenna_type	
coordinate_system	Reference coordinate system.
excitation_type	Excitation type.
frequency	Center frequency.
frequency_unit	Frequency units.
length_unit	Length unit.
material	Horn material.
material_properties	Substrate material properties.

continues on next page

Table 37 – continued from previous page

name	Antenna name.
origin	Antenna origin.
outer_boundary	Outer boundary.

ansys.aedt.toolkits.antenna.backend.antenna_models.horn.EPlane

class ansys.aedt.toolkits.antenna.backend.antenna_models.horn.**EPlane**(**args*, ***kwargs*)

Manages an E plane horn antenna.

This class is accessible through the app hfss object [**?**].

Parameters

frequency
[float, optional] Center frequency. The default is 10.0.

frequency_unit
[str, optional] Frequency units. The default is "GHz".

material
[str, optional] Horn material. If a material is not defined, a new material, parametrized, is defined. The default is "pec".

outer_boundary
[str, optional] Boundary type to use. The default is None. Options are "FEBI", "PML", "Radiation", and None.

length_unit
[str, optional] Length units. The default is "mm".

parametrized
[bool, optional] Whether to create a parametrized antenna. The default is True.

Returns

aedt.toolkits.antenna.EPlaneHorn
E plane horn object.

Notes

Examples

```
>>> from ansys.aedt.toolkits.antenna.backend.antenna_models.horn import EPlane
>>> import ansys.aedt.core
>>> app = ansys.aedt.core.Hfss()
>>> oantenna1 = EPlane(app)
>>> oantenna1.frequency = 12.0
>>> oantenna1.model_hfss()
>>> oantenna1.setup_hfss()
>>> oantenna2 = EPlane(app, origin=[0.2, 0.5, 0])
>>> oantenna2.model_hfss()
>>> oantenna2.setup_hfss()
>>> app.release_desktop(False, False)
```

__init__(**args*, ***kwargs*)

Methods

`__init__`(*args, **kwargs)	
`create_3dcomponent`([component_file, ...])	Create a 3D component of the antenna.
`create_lattice_pair`([lattice_height, ...])	Create a lattice pair box.
`duplicate_along_line`(vector[, num_clones])	Duplicate the object along a line.
`init_model`()	Create a radiation boundary.
`model_disco`()	Model in PyDiscovery.
`model_hfss`()	Draw E plane horn antenna.
`set_variables_in_hfss`([not_used])	Create HFSS design variables.
`setup_disco`()	Set up model in PyDiscovery.
`setup_hfss`()	Set up an antenna in HFSS.
`synthesis`()	Antenna synthesis.
`update_synthesis_parameters`(new_params)	Update the synthesis parameter from the antenna list.

Attributes

`antenna_type`	
`coordinate_system`	Reference coordinate system.
`excitation_type`	Excitation type.
`frequency`	Center frequency.
`frequency_unit`	Frequency units.
`length_unit`	Length unit.
`material`	Horn material.
`material_properties`	Substrate material properties.
`name`	Antenna name.
`origin`	Antenna origin.
`outer_boundary`	Outer boundary.

ansys.aedt.toolkits.antenna.backend.antenna_models.horn.HPlane

class ansys.aedt.toolkits.antenna.backend.antenna_models.horn.**HPlane**(**args*, ***kwargs*)

Manages an H plane horn antenna.

This class is accessible through the app hfss object [**?**].

Parameters

frequency
[float, optional] Center frequency. The default is `10.0`.

frequency_unit
[str, optional] Frequency units. The default is `"GHz"`.

material
[str, optional] Horn material. If a material is not defined, a new material, `parametrized`, is defined. The default is `"pec"`.

outer_boundary
[str, optional] Boundary type to use. The default is `None`. Options are `"FEBI"`, `"PML"`, `"Radiation"`, and `None`.

length_unit
[str, optional] Length units. The default is `"mm"`.

parametrized
[bool, optional] Whether to create a parametrized antenna. The default is True.

Returns

aedt.toolkits.antenna.HPlaneHorn
H plane horn object.

Notes

Examples

```
>>> from ansys.aedt.toolkits.antenna.backend.antenna_models.horn import HPlane
>>> import ansys.aedt.core
>>> app = ansys.aedt.core.Hfss()
>>> oantenna1 = HPlane(app)
>>> oantenna1.frequency = 12.0
>>> oantenna1.model_hfss()
>>> oantenna1.setup_hfss()
>>> oantenna2 = HPlane(app, origin=[0.2, 0.5, 0])
>>> oantenna2.model_hfss()
>>> oantenna2.setup_hfss()
>>> app.release_desktop(False, False)
```

__init__(**args*, ***kwargs*)

Methods

__init__(*args, **kwargs)	
create_3dcomponent([component_file, ...])	Create a 3D component of the antenna.
create_lattice_pair([lattice_height, ...])	Create a lattice pair box.
duplicate_along_line(vector[, num_clones])	Duplicate the object along a line.
init_model()	Create a radiation boundary.
model_disco()	Model in PyDiscovery.
model_hfss()	Draw H plane horn antenna.
set_variables_in_hfss([not_used])	Create HFSS design variables.
setup_disco()	Set up model in PyDiscovery.
setup_hfss()	Set up an antenna in HFSS.
synthesis()	Antenna synthesis.
update_synthesis_parameters(new_params)	Update the synthesis parameter from the antenna list.

Attributes

antenna_type	
coordinate_system	Reference coordinate system.
excitation_type	Excitation type.
frequency	Center frequency.
frequency_unit	Frequency units.
length_unit	Length unit.
material	Horn material.
material_properties	Substrate material properties.
name	Antenna name.

continues on next page

Table 41 – continued from previous page

origin	Antenna origin.
outer_boundary	Outer boundary.

ansys.aedt.toolkits.antenna.backend.antenna_models.horn.Pyramidal

class ansys.aedt.toolkits.antenna.backend.antenna_models.horn.**Pyramidal**(**args*, ***kwargs*)

Manages a pyramidal horn antenna.

This class is accessible through the app hfss object [**?**].

Parameters

frequency
[float, optional] Center frequency. The default is 10.0.

frequency_unit
[str, optional] Frequency units. The default is "GHz".

material
[str, optional] Horn material. If a material is not defined, a new material, parametrized, is defined. The default is "pec".

outer_boundary
[str, optional] Boundary type to use. The default is None. Options are "FEBI", "PML", "Radiation", and None.

length_unit
[str, optional] Length units. The default is "mm".

parametrized
[bool, optional] Whether to create a parametrized antenna. The default is True.

Returns

aedt.toolkits.antenna.Pyramidal
Pyramidal horn object.

Notes

Examples

```
>>> from ansys.aedt.toolkits.antenna.backend.antenna_models.horn import Pyramidal
>>> import ansys.aedt.core
>>> app = ansys.aedt.core.Hfss()
>>> oantenna1 = Pyramidal(app)
>>> oantenna1.frequency = 12.0
>>> oantenna1.model_hfss()
>>> oantenna1.setup_hfss()
>>> oantenna2 = Pyramidal(app, origin=[0.2, 0.5, 0])
>>> oantenna2.model_hfss()
>>> oantenna2.setup_hfss()
>>> app.release_desktop(False, False)
```

__init__(**args*, ***kwargs*)

Methods

`__init__`(*args, **kwargs)	
`create_3dcomponent`([component_file, ...])	Create a 3D component of the antenna.
`create_lattice_pair`([lattice_height, ...])	Create a lattice pair box.
`duplicate_along_line`(vector[, num_clones])	Duplicate the object along a line.
`init_model`()	Create a radiation boundary.
`model_disco`()	Model in PyDiscovery.
`model_hfss`()	Draw pyramidal horn antenna.
`set_variables_in_hfss`([not_used])	Create HFSS design variables.
`setup_disco`()	Set up model in PyDiscovery.
`setup_hfss`()	Set up an antenna in HFSS.
`synthesis`()	Antenna synthesis.
`update_synthesis_parameters`(new_params)	Update the synthesis parameter from the antenna list.

Attributes

`antenna_type`	
`coordinate_system`	Reference coordinate system.
`excitation_type`	Excitation type.
`frequency`	Center frequency.
`frequency_unit`	Frequency units.
`length_unit`	Length unit.
`material`	Horn material.
`material_properties`	Substrate material properties.
`name`	Antenna name.
`origin`	Antenna origin.
`outer_boundary`	Outer boundary.

ansys.aedt.toolkits.antenna.backend.antenna_models.horn.PyramidalRidged

class ansys.aedt.toolkits.antenna.backend.antenna_models.horn.**PyramidalRidged**(**args*, ***kwargs*)

Manages a pyramidal ridged horn antenna.

This class is accessible through the app hfss object [**?**].

Parameters

frequency
[`float`, `optional`] Center frequency. The default is `10.0`.

frequency_unit
[`str`, `optional`] Frequency units. The default is `"GHz"`.

material
[`str`, `optional`] Horn material. If a material is not defined, a new material, `parametrized`, is defined. The default is `"pec"`.

outer_boundary
[`str`, `optional`] Boundary type to use. The default is `None`. Options are `"FEBI"`, `"PML"`, `"Radiation"`, and `None`.

length_unit
[`str`, `optional`] Length units. The default is `"mm"`.

parametrized
[bool, optional] Whether to create a parametrized antenna. The default is True.

Returns

aedt.toolkits.antenna.PyramidalRidged
Pyramidal ridged horn object.

Notes

Examples

```
>>> from ansys.aedt.toolkits.antenna.backend.antenna_models.horn import␣
↪PyramidalRidged
>>> import ansys.aedt.core
>>> app = ansys.aedt.core.Hfss()
>>> oantenna1 = PyramidalRidged(app)
>>> oantenna1.frequency = 12.0
>>> oantenna1.model_hfss()
>>> oantenna1.setup_hfss()
>>> oantenna2 = PyramidalRidged(app, origin=[0.2, 0.5, 0])
>>> oantenna2.model_hfss()
>>> oantenna2.setup_hfss()
>>> app.release_desktop(False, False)
```

__init__(**args*, ***kwargs*)

Methods

__init__(*args, **kwargs)	
create_3dcomponent([component_file, ...])	Create a 3D component of the antenna.
create_lattice_pair([lattice_height, ...])	Create a lattice pair box.
duplicate_along_line(vector[, num_clones])	Duplicate the object along a line.
init_model()	Create a radiation boundary.
model_disco()	Model in PyDiscovery.
model_hfss()	Draw conical horn antenna.
set_variables_in_hfss([not_used])	Create HFSS design variables.
setup_disco()	Set up model in PyDiscovery.
setup_hfss()	Set up an antenna in HFSS.
synthesis()	Antenna synthesis.
update_synthesis_parameters(new_params)	Update the synthesis parameter from the antenna list.

Attributes

antenna_type	
coordinate_system	Reference coordinate system.
excitation_type	Excitation type.
frequency	Center frequency.
frequency_unit	Frequency units.
length_unit	Length unit.
material	Horn material.
material_properties	Substrate material properties.

continues on next page

Table 45 – continued from previous page

name	Antenna name.
origin	Antenna origin.
outer_boundary	Outer boundary.

ansys.aedt.toolkits.antenna.backend.antenna_models.horn.QuadRidged

class ansys.aedt.toolkits.antenna.backend.antenna_models.horn.**QuadRidged**(**args*, ***kwargs*)

Manages a quad-ridged horn antenna.

This class is accessible through the app hfss object [**?**], [**?**], [**?**].

Parameters

frequency
[float, optional] Center frequency. The default is 10.0.

frequency_unit
[str, optional] Frequency units. The default is "GHz".

material
[str, optional] Horn material. If a material is not defined, a new material, parametrized, is defined. The default is "pec".

outer_boundary
[str, optional] Boundary type to use. The default is None. Options are "FEBI", "PML", "Radiation", and None.

length_unit
[str, optional] Length units. The default is "mm".

parametrized
[bool, optional] Whether to create a parametrized antenna. The default is True.

Returns

aedt.toolkits.antenna.PyramidalRidged
Pyramidal ridged horn object.

Notes

Examples

```
>>> from ansys.aedt.toolkits.antenna.backend.antenna_models.horn import QuadRidged
>>> import ansys.aedt.core
>>> app = ansys.aedt.core.Hfss()
>>> oantenna1 = QuadRidged(app)
>>> oantenna1.frequency = 12.0
>>> oantenna1.model_hfss()
>>> oantenna1.setup_hfss()
>>> oantenna2 = QuadRidged(app, origin=[0.2, 0.5, 0])
>>> oantenna2.model_hfss()
>>> oantenna2.setup_hfss()
>>> app.release_desktop(False, False)
```

__init__(**args*, ***kwargs*)

Methods

__init__(*args, **kwargs)	
create_3dcomponent([component_file, ...])	Create a 3D component of the antenna.
create_lattice_pair([lattice_height, ...])	Create a lattice pair box.
duplicate_along_line(vector[, num_clones])	Duplicate the object along a line.
init_model()	Create a radiation boundary.
model_disco()	Model in PyDiscovery.
model_hfss()	Draw conical horn antenna.
set_variables_in_hfss([not_used])	Create HFSS design variables.
setup_disco()	Set up model in PyDiscovery.
setup_hfss()	Set up an antenna in HFSS.
synthesis()	Antenna synthesis.
update_synthesis_parameters(new_params)	Update the synthesis parameter from the antenna list.

Attributes

antenna_type	
coordinate_system	Reference coordinate system.
excitation_type	Excitation type.
frequency	Center frequency.
frequency_unit	Frequency units.
length_unit	Length unit.
material	Horn material.
material_properties	Substrate material properties.
name	Antenna name.
origin	Antenna origin.
outer_boundary	Outer boundary.

You must use these methods from PyAEDT as shown in this example:

```
import ansys.aedt.core.Hfss

from ansys.aedt.toolkits.antenna.backend.antenna_models.horn import Conical

aedtapp = Hfss()

# Create antenna
oantenna1 = Conical(app)
oantenna1.frequency = 12.0
oantenna1.model_hfss()
oantenna1.setup_hfss()
...
aedtapp.release_desktop()
```

3.2.7 Patch

This page lists the classes available for patch antennas:

RectangularPatchEdge(*args, **kwargs)	Manages a rectangular patch edge antenna.

continues on next page

Table 48 – continued from previous page

RectangularPatchProbe(*args, **kwargs)	Manages a rectangular patch antenna with a coaxial probe.
RectangularPatchInset(*args, **kwargs)	Manages a rectangular patch antenna inset fed.

ansys.aedt.toolkits.antenna.backend.antenna_models.patch.RectangularPatchEdge

class ansys.aedt.toolkits.antenna.backend.antenna_models.patch.**RectangularPatchEdge**(**args*, ***kwargs*)

Manages a rectangular patch edge antenna.

This class is accessible through the `Hfss` object [**?**].

Parameters

frequency
[`float`, `optional`] Center frequency. The default is `10.0`.

frequency_unit
[`str`, `optional`] Frequency units. The default is `"GHz"`.

material
[`str`, `optional`] Substrate material. If the material is not defined, a new material, `parametrized`, is created. The default is `"FR4_epoxy"`.

outer_boundary
[`str`, `optional`] Boundary type to use. The default is `None`. Options are `"FEBI"`, `"PML"`, `"Radiation"`, and `None`.

length_unit
[`str`, `optional`] Length units. The default is `"mm"`.

substrate_height
[`float`, `optional`] Substrate height. The default is `1.575`.

parametrized
[bool, `optional`] Whether to create a parametrized antenna. The default is `True`.

Returns

`aedt.toolkits.antenna.RectangularPatchEdge`
Patch antenna object.

Notes

Examples

```
>>> from ansys.aedt.toolkits.antenna.backend.antenna_models.patch import␣
↪RectangularPatchEdge
>>> import ansys.aedt.core
>>> app = ansys.aedt.core.Hfss()
>>> oantenna1 = RectangularPatchEdge(app)
>>> oantenna1.frequency = 12.0
>>> oantenna1.model_hfss()
>>> oantenna1.setup_hfss()
>>> app.release_desktop(False, False)
```

__init__(**args*, ***kwargs*)

Methods

`__init__`(*args, **kwargs)	
`create_3dcomponent`([component_file, ...])	Create a 3D component of the antenna.
`create_lattice_pair`([lattice_height, ...])	Create a lattice pair box.
`duplicate_along_line`(vector[, num_clones])	Duplicate the object along a line.
`init_model`()	Create a radiation boundary.
`model_disco`()	Model in PyDiscovery.
`model_hfss`()	Draw a rectangular patch edge antenna inset fed.
`set_variables_in_hfss`([not_used])	Create HFSS design variables.
`setup_disco`()	Set up the model in PyDiscovery.
`setup_hfss`()	Set up a patch antenna in HFSS.
`synthesis`()	Antenna synthesis.
`update_synthesis_parameters`(new_params)	Update the synthesis parameter from the antenna list.

Attributes

`antenna_type`	
`coordinate_system`	Reference coordinate system.
`element_1_port_phase`	
`element_1_rotation_angle`	
`element_2_port_phase`	
`element_2_rotation_angle`	
`element_3_port_phase`	
`element_3_rotation_angle`	
`element_4_port_phase`	
`element_4_rotation_angle`	
`excitation_type`	Excitation type.
`feed_rotation_angle`	
`frequency`	Center frequency.
`frequency_unit`	Frequency units.
`length_unit`	Length unit.
`material`	Substrate material.
`material_properties`	Substrate material properties.
`name`	Antenna name.
`number_of_patches_x`	
`number_of_patches_y`	
`origin`	Antenna origin.
`outer_boundary`	Outer boundary.
`substrate_height`	Substrate height.

ansys.aedt.toolkits.antenna.backend.antenna_models.patch.RectangularPatchProbe

class ansys.aedt.toolkits.antenna.backend.antenna_models.patch.**RectangularPatchProbe**(**args*, ***kwargs*)

Manages a rectangular patch antenna with a coaxial probe.

This class is accessible through the `Hfss` object [**?**].

Parameters

frequency
[`float`, `optional`] Center frequency. The default is `10.0`.

frequency_unit
[str, optional] Frequency units. The default is "GHz".

material
[str, optional] Substrate material. If the material is not defined, a new material, parametrized, is created. The default is "FR4_epoxy".

outer_boundary
[str, optional] Boundary type to use. The default is None. Options are "FEBI", "PML", "Radiation", and None.

length_unit
[str, optional] Length units. The default is "mm".

substrate_height
[float, optional] Substrate height. The default is 1.575.

parametrized
[bool, optional] Whether to create a parametrized antenna. The default is True.

Returns

aedt.toolkits.antenna.RectangularPatchProbe
Patch antenna object.

Notes

Examples

```
>>> from ansys.aedt.toolkits.antenna.backend.antenna_models.patch import␣
↪RectangularPatchProbe
>>> import ansys.aedt.core
>>> app = ansys.aedt.core.Hfss()
>>> oantenna1 = RectangularPatchProbe(app)
>>> oantenna1.frequency = 12.0
>>> oantenna1.model_hfss()
>>> oantenna1.setup_hfss()
>>> app.release_desktop(False, False)
```

__init__(**args*, ***kwargs*)

Methods

__init__(*args, **kwargs)	
create_3dcomponent([component_file, ...])	Create a 3D component of the antenna.
create_lattice_pair([lattice_height, ...])	Create a lattice pair box.
duplicate_along_line(vector[, num_clones])	Duplicate the object along a line.
init_model()	Create a radiation boundary.
model_disco()	Model in PyDiscovery.
model_hfss()	Draw rectangular patch antenna with coaxial probe.
set_variables_in_hfss([not_used])	Create HFSS design variables.
setup_disco()	Set up the model in PyDiscovery.
setup_hfss()	Set up a patch antenna in HFSS.
synthesis()	Antenna synthesis.
update_synthesis_parameters(new_params)	Update the synthesis parameter from the antenna list.

Attributes

antenna_type	
coordinate_system	Reference coordinate system.
element_1_port_phase	
element_1_rotation_angle	
element_2_port_phase	
element_2_rotation_angle	
element_3_port_phase	
element_3_rotation_angle	
element_4_port_phase	
element_4_rotation_angle	
excitation_type	Excitation type.
feed_rotation_angle	
frequency	Center frequency.
frequency_unit	Frequency units.
length_unit	Length unit.
material	Substrate material.
material_properties	Substrate material properties.
name	Antenna name.
number_of_patches_x	
number_of_patches_y	
origin	Antenna origin.
outer_boundary	Outer boundary.
substrate_height	Substrate height.

ansys.aedt.toolkits.antenna.backend.antenna_models.patch.RectangularPatchInset

class ansys.aedt.toolkits.antenna.backend.antenna_models.patch.**RectangularPatchInset**(**args*, ***kwargs*)

Manages a rectangular patch antenna inset fed.

This class is accessible through the `Hfss` object [**?**].

Parameters

frequency
[`float`, `optional`] Center frequency. The default is `10.0`.

frequency_unit
[`str`, `optional`] Frequency units. The default is `"GHz"`.

material
[`str`, `optional`] Substrate material. If the material is not defined, a new material, `parametrized`, is created. The default is `"FR4_epoxy"`.

outer_boundary
[`str`, `optional`] Boundary type to use. The default is `None`. Options are `"FEBI"`, `"PML"`, `"Radiation"`, and `None`.

length_unit
[`str`, `optional`] Length units. The default is `"mm"`.

substrate_height
[`float`, `optional`] Substrate height. The default is `1.575`.

parametrized
[bool, optional] Whether to create a parametrized antenna. The default is True.

Returns

aedt.toolkits.antenna.RectangularPatchInset
Patch antenna object.

Notes

Examples

```
>>> from ansys.aedt.toolkits.antenna.backend.antenna_models.patch import␣
↪RectangularPatchInset
>>> import ansys.aedt.core
>>> app = ansys.aedt.core.Hfss()
>>> oantenna1 = RectangularPatchInset(app)
>>> oantenna1.frequency = 12.0
>>> oantenna1.model_hfss()
>>> oantenna1.setup_hfss()
>>> app.release_desktop(False, False)
```

__init__(**args*, ***kwargs*)

Methods

__init__(*args, **kwargs)	
create_3dcomponent([component_file, ...])	Create a 3D component of the antenna.
create_lattice_pair([lattice_height, ...])	Create a lattice pair box.
duplicate_along_line(vector[, num_clones])	Duplicate the object along a line.
init_model()	Create a radiation boundary.
model_disco()	Model in PyDiscovery.
model_hfss()	Draw a rectangular patch antenna inset fed.
set_variables_in_hfss([not_used])	Create HFSS design variables.
setup_disco()	Set up the model in PyDiscovery.
setup_hfss()	Set up a patch antenna in HFSS.
synthesis()	Antenna synthesis.
update_synthesis_parameters(new_params)	Update the synthesis parameter from the antenna list.

Attributes

antenna_type	
coordinate_system	Reference coordinate system.
element_1_port_phase	
element_1_rotation_angle	
element_2_port_phase	
element_2_rotation_angle	
element_3_port_phase	
element_3_rotation_angle	
element_4_port_phase	
element_4_rotation_angle	
excitation_type	Excitation type.

continues on next page

Table 54 – continued from previous page

feed_rotation_angle	
frequency	Center frequency.
frequency_unit	Frequency units.
length_unit	Length unit.
material	Substrate material.
material_properties	Substrate material properties.
name	Antenna name.
number_of_patches_x	
number_of_patches_y	
origin	Antenna origin.
outer_boundary	Outer boundary.
substrate_height	Substrate height.

You must use these methods from PyAEDT as shown in this example:

```
from ansys.aedt.core import Hfss

from ansys.aedt.toolkits.antenna.backend.antenna_models.patch import (
    RectangularPatchEdge,
)

aedtapp = Hfss()

# Create antenna
oantenna1 = RectangularPatchProbe(app)
oantenna1.frequency = 12.0
oantenna1.model_hfss()
oantenna1.setup_hfss()
...
aedtapp.release_desktop()
```

You use the Antenna API at the model level from PyAEDT.

You can create one or more antennas. An antenna is object-oriented. You can synthesis an antenna without AEDT.

This code shows how to synthesis an antenna:

```
# Import backend
from ansys.aedt.toolkits.antenna.backend.models.horn import Conical

# Synthesize antenna
oantenna1 = Conical()
oantenna1.frequency = 12.0
```

This code shows how to synthesize and create a model of an antenna in HFSS:

```
# Import HFSS
from ansys.aedt.core import Hfss

# Import backend
from ansys.aedt.toolkits.antenna.backend.models.horn import Conical

# Synthesize antenna
```

(continues on next page)

(continued from previous page)

```
aedtapp = Hfss()

# Create antenna
oantenna1 = Conical(app)
oantenna1.model_hfss()
oantenna1.setup_hfss()
...
aedtapp.release_desktop()
```

CHAPTER

FOUR

EXAMPLES

End-to-end examples show how to use the API of the AEDT Antenna Toolkit.

4.1 Antenna synthesis

These examples show how to use the different antenna classes:

Copyright (C) 2023 - 2026 Synopsys, Inc. and ANSYS, Inc. All rights reserved. SPDX-License-Identifier: MIT

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the Software), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED AS IS, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

[1]:
```
import tempfile
```

[2]:
```
import ansys.aedt.core
```

[3]:
```
from ansys.aedt.toolkits.antenna.backend.antenna_models.bowtie import BowTieRounded
```

4.1.1 Set AEDT version

Set AEDT version.

[4]:
```
aedt_version = "2026.1"
```

4.1.2 Set non-graphical mode

Set non-graphical mode.

[5]:
```
non_graphical = True
```

4.1.3 Create temporary directory

[6]:
```
temp_dir = tempfile.TemporaryDirectory(suffix="_ansys")
project_name = ansys.aedt.core.generate_unique_project_name(root_name=temp_dir.name,␣
↪project_name="bowtie_example")
```

4.1.4 Create antenna object only for synthesis

Create antenna object.

[7]:
```
oantenna1 = BowTieRounded(None)
print(
    "Arm length: {}{} at {}{}".format(
        str(oantenna1.synthesis_parameters.arm_length.value),
        oantenna1.length_unit,
        oantenna1.frequency,
        oantenna1.frequency_unit,
    )
)
```

```
Arm length: 3.7mm at 10.0GHz
```

4.1.5 Change synthesis frequency

Modify resonance frequency and modify parameters.

[8]:
```
oantenna1.frequency = 12.0
print(
    "Arm length: {}{} at {}{}".format(
        str(oantenna1.synthesis_parameters.arm_length.value),
        oantenna1.length_unit,
        oantenna1.frequency,
        oantenna1.frequency_unit,
    )
)
```

```
Arm length: 3.03mm at 12.0GHz
```

4.1.6 Create an empty HFSS design

Create an empty HFSS design.

[9]:
```
app = ansys.aedt.core.Hfss(project=project_name, version=aedt_version, non_graphical=non_
↪graphical)
```

```
PyAEDT INFO: Python version 3.14.5 (tags/v3.14.5:5607950, May 10 2026, 10:43:50) [MSC v.
↪1944 64 bit (AMD64)].
PyAEDT INFO: PyAEDT version 1.9.0.
PyAEDT INFO: Initializing Desktop session.
PyAEDT INFO: AEDT version 2026.1.
PyAEDT INFO: New AEDT session is starting on gRPC port 54145.
PyAEDT INFO: Starting new AEDT gRPC session on port 54145.
PyAEDT INFO: Launching AEDT server with gRPC transport mode: TransportMode.WNUA
PyAEDT INFO: Electronics Desktop started on gRPC port 54145 after 11.3 seconds.
```

(continues on next page)

(continued from previous page)

```
PyAEDT INFO: AEDT installation Path C:\Program Files\ANSYS Inc\v261\AnsysEM
PyAEDT INFO: 2026.1 version started with process ID 4864.
PyAEDT INFO: Non-graphical mode detected. Disabling Desktop logs.
PyAEDT INFO: Project bowtie_example has been created.
PyAEDT INFO: No design is present. Inserting a new design.
PyAEDT INFO: Added design 'HFSS_265' of type HFSS.
PyAEDT INFO: AEDT objects correctly read
```

4.1.7 Create antenna in HFSS

Create antenna object, change frequency synthesis, create antenna, and set up in HFSS.

[10]:
```
oantenna1 = BowTieRounded(app)
```

```
PyAEDT INFO: Modeler class has been initialized! Elapsed time: 0m 0sec
PyAEDT INFO: Materials class has been initialized! Elapsed time: 0m 0sec
```

[11]:
```
# Create antenna in HFSS.
oantenna1.model_hfss()
```

```
PyAEDT INFO: Parsing design objects. This operation can take time
PyAEDT INFO: Refreshing bodies from Object Info
PyAEDT INFO: Bodies Info Refreshed Elapsed time: 0m 0sec
PyAEDT INFO: 3D Modeler objects parsed. Elapsed time: 0m 0sec
PyAEDT INFO: Union of 2 objects has been executed.
```

[11]:
```
True
```

Create antenna setup.

[12]:
```
oantenna1.setup_hfss()
```

```
PyAEDT INFO: Boundary Perfect E PerfE_Q730G3 has been created.
PyAEDT INFO: Boundary Perfect E PerfE_E0MJR7 has been created.
PyAEDT INFO: Boundary AutoIdentify port_Patch_P8D9R0_1 has been created.
```

[12]:
```
True
```

Change default name.

[13]:
```
oantenna1.name = "MyAmazingAntenna"
```

4.1.8 Create antenna in HFSS

Create antenna object, change origin parameter in the antenna definition, create antenna, and set up in HFSS.

[14]:
```
oantenna2 = BowTieRounded(app, origin=[20, 5, 0], name="MyAntenna")
oantenna2.model_hfss()
oantenna2.setup_hfss()
```

```
PyAEDT INFO: Parsing design objects. This operation can take time
PyAEDT INFO: Refreshing bodies from Object Info
PyAEDT INFO: Bodies Info Refreshed Elapsed time: 0m 0sec
PyAEDT INFO: 3D Modeler objects parsed. Elapsed time: 0m 0sec
PyAEDT INFO: Union of 2 objects has been executed.
```

(continues on next page)

(continued from previous page)

```
PyAEDT INFO: Boundary Perfect E PerfE_LNNDWN has been created.
PyAEDT INFO: Boundary Perfect E PerfE_3G4GUW has been created.
PyAEDT INFO: Boundary AutoIdentify port_MyAntenna_1 has been created.
```

[14]:
```
True
```

4.1.9 Plot HFSS model

Plot geometry with PyVista.

[15]:
```
model = app.plot(show=True)
```

```
PyAEDT INFO: Parsing C:\Users\ansys\AppData\Local\Temp\tmpv9ez7q9r_ansys\pyaedt_prj_NMO\
↪bowtie_example.aedt.
PyAEDT INFO: File C:\Users\ansys\AppData\Local\Temp\tmpv9ez7q9r_ansys\pyaedt_prj_NMO\
↪bowtie_example.aedt correctly loaded. Elapsed time: 0m 0sec
PyAEDT INFO: aedt file load time 0.019503116607666016
PyAEDT INFO: PostProcessor class has been initialized! Elapsed time: 0m 0sec
PyAEDT INFO: PostProcessor class has been initialized! Elapsed time: 0m 0sec
PyAEDT INFO: Post class has been initialized! Elapsed time: 0m 0sec
```

```
C:\actions-runner\_work\pyaedt-toolkits-antenna\pyaedt-toolkits-antenna\.venv\Lib\site-
↪packages\ansys\aedt\core\visualization\plot\pyvista.py:1366: PyVistaDeprecationWarning:
↪ ``Plotter.button_widgets`` is deprecated; use ``Plotter.widgets.button_widgets``␣
↪instead.
  while len(self.plot.button_widgets) > 1:
```

```
EmbeddableWidget(value='<iframe srcdoc="<!doctype html>\n<html lang=&quot;en&quot;>\n
↪<head>\n    <meta chars
```

4.1.10 Release AEDT

Release AEDT.

[16]:
```
app.release_desktop(True, True)
```

```
PyAEDT INFO: Desktop has been released and closed.
```

[16]:
```
True
```

4.1.11 Clean temporary directory

[17]:
```
temp_dir.cleanup()
```

4.2 Antenna toolkit

These examples show how to use the `ToolkitBackend` class:

Copyright (C) 2023 - 2026 Synopsys, Inc. and ANSYS, Inc. All rights reserved. SPDX-License-Identifier: MIT

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the Software), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED AS IS, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

[1]:
```
from pathlib import Path
import sys
import tempfile
```

[2]:
```
from ansys.aedt.core import generate_unique_project_name
from ansys.aedt.core.visualization.advanced.farfield_visualization import FfdSolutionData
from ansys.aedt.core.visualization.plot.pyvista import ModelPlotter
import pyvista as pv
```

[3]:
```
from ansys.aedt.toolkits.antenna.backend.api import ToolkitBackend
from ansys.aedt.toolkits.antenna.backend.models import properties
```

4.2.1 Set AEDT version

Set AEDT version.

[4]:
```
aedt_version = "2026.1"
```

4.2.2 Set non-graphical mode

Set non-graphical mode.

[5]:
```
non_graphical = False
```

4.2.3 Set number of cores

[6]:
```
num_cores = 4
```

4.2.4 Create temporary directory

[7]:
```
temp_dir = tempfile.TemporaryDirectory(suffix="_ansys")
project_name = generate_unique_project_name(root_name=temp_dir.name, project_name=
↪"antenna_toolkit")
```

4.2.5 Set default properties

[8]:
```
properties.aedt_version = aedt_version
properties.non_graphical = non_graphical
properties.active_project = project_name
```

4.2.6 Initialize toolkit

Initialize the toolkit.

[9]:
```
toolkit_api = ToolkitBackend()
```

4.2.7 Get available_antennas

[10]:
```
print(toolkit_api.available_antennas)
```

```
['BowTieNormal', 'BowTieRounded', 'BowTieSlot', 'Archimedean', 'Log', 'Sinuous',
↪'GPSPatchCeramic', 'PlanarDipole', 'WireDipole', 'AxialMode', 'AxialModeTaper',
↪'NormalMode', 'QuadrifilarOpen', 'QuadrifilarShort', 'Conical', 'Corrugated',
↪'Elliptical', 'EPlane', 'HPlane', 'Pyramidal', 'PyramidalRidged', 'QuadRidged',
↪'LogPeriodicArray', 'LogPeriodicToothed', 'LogPeriodicTrapezoidal', 'Bicone', 'Discone
↪', 'BladeAntenna', 'CircularDiscMonopole', 'EllipticalBaseStripMonopole',
↪'VerticalTrapezoidalMonopole', 'WireMonopole', 'EllipticalEdge', 'EllipticalInset',
↪'EllipticalProbe', 'MbyNPatchArray', 'RectangularPatchEdge', 'RectangularPatchInset',
↪'RectangularPatchProbe', 'SeqRotated2Patch', 'PlanarInvertedF', 'ShortingPin',
↪'ShortingPlate', 'PlanarArchimedean', 'PlanarArchimedeanCavity', 'PlanarLog',
↪'PlanarLogCavity', 'PlanarSinuous', 'PlanarSinuousCavity', 'SlotCavityBackedArray',
↪'SlotGap', 'SlotMicrostrip', 'SlotTBar', 'Vivaldi', 'VivaldiStepped',
↪'CircularWaveguide', 'RectangularWaveguide', 'RectangularWaveguideSlotArray',
↪'QuasiYagi', 'WireYagiUda']
```

4.2.8 Get default properties

[11]:
```
backend_properties = toolkit_api.get_properties()
frequency = backend_properties["antenna"]["synthesis"]["frequency"]
frequency_units = backend_properties["antenna"]["synthesis"]["frequency_unit"]
length_unit = backend_properties["antenna"]["synthesis"]["length_unit"]
```

4.2.9 Modify default length units

[12]:
```
properties.antenna.synthesis.length_unit = "cm"
properties.antenna.synthesis.substrate_height = 0.1
```

4.2.10 Create antenna object only for synthesis

Create antenna object.

[13]:
```
antenna_parameters_1 = toolkit_api.get_antenna("RectangularPatchProbe", synth_only=True)
```

```
INFO - AEDT is released.
```

[14]:
```
print(
    "Patch X length: {}{} at {}{}".format(
        str(antenna_parameters_1["patch_x"]),
        length_unit,
        frequency,
        frequency_units,
    )
)
```

```
Patch X length: 0.912871mm at 10.0GHz
```

4.2.11 Change synthesis frequency

Modify resonance frequency and modify parameters with the `set_properties()` method.

[15]:
```
new_frequency1 = 12.0
new_properties = {"frequency": new_frequency1}
toolkit_api.set_properties(new_properties)
```

```
INFO - Updating internal properties.
DEBUG - Updating 'frequency' with value 12.0
DEBUG - Properties were updated successfully.
```

[15]:
```
(True, 'Properties were updated successfully.')
```

[16]:
```
antenna_parameters_2 = toolkit_api.get_antenna("RectangularPatchProbe", synth_only=True)
```

```
INFO - AEDT is released.
```

[17]:
```
print(
    "Patch X length: {}{} at {}{}".format(
        str(antenna_parameters_2["patch_x"]),
        length_unit,
        new_frequency1,
        frequency_units,
    )
)
```

```
Patch X length: 0.760726mm at 12.0GHz
```

4.2.12 Change synthesis frequency

Modify resonance frequency with properties directly.

[18]:
```
new_frequency2 = 15.0
properties.antenna.synthesis.frequency = new_frequency2
```

[19]:
```
antenna_parameters_3 = toolkit_api.get_antenna("RectangularPatchProbe", synth_only=True)
```

```
INFO - AEDT is released.
```

[20]:
```
print(
    "Patch X length: {}{} at {}{}".format(
        str(antenna_parameters_3["patch_x"]),
        length_unit,
        new_frequency2,
        frequency_units,
    )
)
```

```
Patch X length: 0.608581mm at 15.0GHz
```

4.2.13 Initialize AEDT

Launch a new AEDT session in a thread.

[21]:
```
thread_msg = toolkit_api.launch_thread(toolkit_api.launch_aedt)
```

```
DEBUG - Starting thread: Toolkit_Thread
DEBUG - Toolkit is not connected to AEDT.
DEBUG - Launching AEDT
```

```
PyAEDT INFO: Python version 3.14.5 (tags/v3.14.5:5607950, May 10 2026, 10:43:50) [MSC v.
↪1944 64 bit (AMD64)].
```

4.2.14 Wait for the toolkit thread to be idle

Wait for the toolkit thread to be idle and ready to accept a new task.

[22]:
```
idle = toolkit_api.wait_to_be_idle()
if not idle:
    print("AEDT not initialized.")
    sys.exit()
```

```
PyAEDT INFO: AEDT version 2026.1.
PyAEDT INFO: New AEDT session is starting on gRPC port 53725.
PyAEDT INFO: Starting new AEDT gRPC session on port 53725.
PyAEDT INFO: Launching AEDT server with gRPC transport mode: TransportMode.WNUA
```

```
DEBUG - Toolkit is busy and processing a task.
DEBUG - Toolkit is busy and processing a task.
DEBUG - Toolkit is busy and processing a task.
DEBUG - Toolkit is busy and processing a task.
DEBUG - Toolkit is busy and processing a task.
DEBUG - Toolkit is busy and processing a task.
DEBUG - Toolkit is busy and processing a task.
DEBUG - Toolkit is busy and processing a task.
DEBUG - Toolkit is busy and processing a task.
DEBUG - Toolkit is busy and processing a task.
DEBUG - Toolkit is busy and processing a task.
```

```
PyAEDT INFO: Electronics Desktop started on gRPC port 53725 after 11.2 seconds.
PyAEDT INFO: AEDT installation Path C:\Program Files\ANSYS Inc\v261\AnsysEM
PyAEDT INFO: 2026.1 version started with process ID 328.
PyAEDT INFO: Non-graphical mode detected. Disabling Desktop logs.
```

```
DEBUG - AEDT launched
DEBUG - Grpc port 53725.
```

```
PyAEDT INFO: Desktop has been released.
```

```
INFO - AEDT is released.
DEBUG - AEDT released and properties loaded
DEBUG - Toolkit is busy and processing a task.
DEBUG - Toolkit is idle and ready to accept a new task.
```

4.2.15 Connect to HFSS design

Create an HFSS design.

[23]:
```
toolkit_api.connect_design("HFSS")
```

```
DEBUG - Toolkit is not connected to AEDT.
DEBUG - Connecting AEDT.
```

```
PyAEDT INFO: Python version 3.14.5 (tags/v3.14.5:5607950, May 10 2026, 10:43:50) [MSC v.
↪1944 64 bit (AMD64)].
PyAEDT INFO: PyAEDT version 1.9.0.
PyAEDT INFO: Initializing Desktop session.
PyAEDT INFO: AEDT version 2026.1.
PyAEDT INFO: Port 53725 session has been found.
PyAEDT INFO: AEDT installation Path C:\Program Files\ANSYS Inc\v261\AnsysEM
PyAEDT INFO: Connected to AEDT gRPC session on port 53725.
PyAEDT INFO: Non-graphical mode detected. Disabling Desktop logs.
```

```
DEBUG - Toolkit is connected to AEDT.
```

```
PyAEDT INFO: Python version 3.14.5 (tags/v3.14.5:5607950, May 10 2026, 10:43:50) [MSC v.
↪1944 64 bit (AMD64)].
PyAEDT INFO: PyAEDT version 1.9.0.
PyAEDT INFO: Returning found Desktop session with PID 328!
PyAEDT INFO: Project antenna_toolkit has been created.
PyAEDT INFO: Added design 'HFSS_CWDT2T' of type HFSS.
PyAEDT INFO: AEDT objects correctly read
PyAEDT INFO: Project antenna_toolkit Saved correctly
PyAEDT INFO: Active Design set to HFSS_CWDT2T
```

```
DEBUG - Project name: antenna_toolkit
INFO - Updating internal properties.
DEBUG - Updating 'project_list' with value ['C:\\Users\\ansys\\AppData\\Local\\Temp\\
↪tmp6v9t96dr_ansys\\pyaedt_prj_FK2\\antenna_toolkit.aedt']
DEBUG - Updating 'active_design' with value HFSS_CWDT2T
DEBUG - Updating 'active_project' with value C:\Users\ansys\AppData\Local\Temp\
↪tmp6v9t96dr_ansys\pyaedt_prj_FK2\antenna_toolkit.aedt
DEBUG - Updating 'design_list' with value {'antenna_toolkit': ['HFSS_CWDT2T']}
DEBUG - Properties were updated successfully.
INFO - Toolkit is connected to AEDT design.
```

[23]:
```
True
```

4.2.16 Create setup when antenna is created

Set `create_setup` property.

[24]:
```
properties.antenna.setup.create_setup = True
properties.antenna.synthesis.outer_boundary = "Radiation"
```

4.2.17 Create antenna in HFSS

Create antenna and set up in HFSS.

[25]:
```
antenna_parameter = toolkit_api.get_antenna("RectangularPatchProbe")
```

```
PyAEDT INFO: Modeler class has been initialized! Elapsed time: 0m 0sec
PyAEDT INFO: Materials class has been initialized! Elapsed time: 0m 0sec
PyAEDT INFO: Open Region correctly created.
PyAEDT INFO: Project antenna_toolkit Saved correctly
PyAEDT INFO: Parsing design objects. This operation can take time
PyAEDT INFO: Refreshing bodies from Object Info
PyAEDT INFO: Bodies Info Refreshed Elapsed time: 0m 0sec
PyAEDT INFO: 3D Modeler objects parsed. Elapsed time: 0m 0sec
PyAEDT INFO: Boundary Perfect E PerfE_7JJSZP has been created.
PyAEDT INFO: Boundary Perfect E PerfE_E8X2TF has been created.
PyAEDT INFO: Boundary Perfect E PerfE_BFLSUB has been created.
PyAEDT INFO: Boundary AutoIdentify port_Patch_WUL65U_1 has been created.
PyAEDT INFO: Project antenna_toolkit Saved correctly
PyAEDT INFO: Desktop has been released.
```

```
INFO - AEDT is released.
```

4.2.18 Plot antenna model

[26]:
```
toolkit_api.connect_design("HFSS")
# Save the plot to display it
plot_obj = toolkit_api.aedtapp.plot(show=False, output_file=str(Path(temp_dir.name) /
↪"antenna_model.png"))
toolkit_api.release_aedt(False, False)
```

```
DEBUG - Toolkit is not connected to AEDT.
DEBUG - Connecting AEDT.
```

```
PyAEDT INFO: Python version 3.14.5 (tags/v3.14.5:5607950, May 10 2026, 10:43:50) [MSC v.
↪1944 64 bit (AMD64)].
PyAEDT INFO: PyAEDT version 1.9.0.
PyAEDT INFO: Initializing Desktop session.
PyAEDT INFO: AEDT version 2026.1.
PyAEDT INFO: Port 53725 session has been found.
PyAEDT INFO: AEDT installation Path C:\Program Files\ANSYS Inc\v261\AnsysEM
PyAEDT INFO: Connected to AEDT gRPC session on port 53725.
PyAEDT INFO: Non-graphical mode detected. Disabling Desktop logs.
```

```
DEBUG - Toolkit is connected to AEDT.
```

```
PyAEDT INFO: Python version 3.14.5 (tags/v3.14.5:5607950, May 10 2026, 10:43:50) [MSC v.
↪1944 64 bit (AMD64)].
PyAEDT INFO: PyAEDT version 1.9.0.
PyAEDT INFO: Returning found Desktop session with PID 328!
PyAEDT INFO: Project antenna_toolkit set to active.
PyAEDT INFO: Active Design set to HFSS_CWDT2T
PyAEDT INFO: AEDT objects correctly read
```

```
INFO - Toolkit is connected to AEDT design.
```

```
PyAEDT INFO: Parsing C:\Users\ansys\AppData\Local\Temp\tmp6v9t96dr_ansys\pyaedt_prj_FK2\
↪antenna_toolkit.aedt.
PyAEDT INFO: File C:\Users\ansys\AppData\Local\Temp\tmp6v9t96dr_ansys\pyaedt_prj_FK2\
↪antenna_toolkit.aedt correctly loaded. Elapsed time: 0m 0sec
PyAEDT INFO: aedt file load time 0.03898310661315918
```

(continues on next page)

(continued from previous page)

```
PyAEDT INFO: PostProcessor class has been initialized! Elapsed time: 0m 0sec
PyAEDT INFO: PostProcessor class has been initialized! Elapsed time: 0m 0sec
PyAEDT INFO: Post class has been initialized! Elapsed time: 0m 0sec
PyAEDT INFO: Modeler class has been initialized! Elapsed time: 0m 0sec
```

```
C:\actions-runner\_work\pyaedt-toolkits-antenna\pyaedt-toolkits-antenna\.venv\Lib\site-
↪packages\ansys\aedt\core\visualization\plot\pyvista.py:1366: PyVistaDeprecationWarning:
↪ ``Plotter.button_widgets`` is deprecated; use ``Plotter.widgets.button_widgets``␣
↪instead.
  while len(self.plot.button_widgets) > 1:
```

```
EmbeddableWidget(value='<iframe srcdoc="<!doctype html>\n<html lang=&quot;en&quot;>\n
↪<head>\n    <meta chars
```

```
PyAEDT INFO: Desktop has been released.
```

```
INFO - AEDT is released.
```

[26]:
```
True
```

4.2.19 Try to create antenna

The AEDT Antenna Toolkit API does not allow the creation of more than one antenna. However, you can use the antenna models API to create more than one antenna.

[27]:
```
new_antenna = toolkit_api.get_antenna("BowTie")
```

```
DEBUG - Antenna is already created.
```

[28]:
```
print(new_antenna)
```

```
False
```

4.2.20 Set properties

Move antenna X position

[29]:
```
toolkit_api.update_hfss_parameters("pos_x", "20")
```

```
DEBUG - Toolkit is not connected to AEDT.
DEBUG - Connecting AEDT.
```

```
PyAEDT INFO: Python version 3.14.5 (tags/v3.14.5:5607950, May 10 2026, 10:43:50) [MSC v.
↪1944 64 bit (AMD64)].
PyAEDT INFO: PyAEDT version 1.9.0.
PyAEDT INFO: Initializing Desktop session.
PyAEDT INFO: AEDT version 2026.1.
PyAEDT INFO: Port 53725 session has been found.
PyAEDT INFO: AEDT installation Path C:\Program Files\ANSYS Inc\v261\AnsysEM
PyAEDT INFO: Connected to AEDT gRPC session on port 53725.
PyAEDT INFO: Non-graphical mode detected. Disabling Desktop logs.
```

```
DEBUG - Toolkit is connected to AEDT.
```

```
PyAEDT INFO: Python version 3.14.5 (tags/v3.14.5:5607950, May 10 2026, 10:43:50) [MSC v.
↪1944 64 bit (AMD64)].
PyAEDT INFO: PyAEDT version 1.9.0.
```

(continues on next page)

(continued from previous page)

```
PyAEDT INFO: Returning found Desktop session with PID 328!
PyAEDT INFO: Project antenna_toolkit set to active.
PyAEDT INFO: Active Design set to HFSS_CWDT2T
PyAEDT INFO: AEDT objects correctly read
```

```
INFO - Toolkit is connected to AEDT design.
```

```
PyAEDT INFO: Desktop has been released.
```

```
INFO - AEDT is released.
```

```
[29]: True
```

4.2.21 Set properties

Move antenna X position to origin

```
[30]: toolkit_api.update_hfss_parameters("pos_x", "0")
```

```
DEBUG - Toolkit is not connected to AEDT.
DEBUG - Connecting AEDT.
```

```
PyAEDT INFO: Python version 3.14.5 (tags/v3.14.5:5607950, May 10 2026, 10:43:50) [MSC v.
↪1944 64 bit (AMD64)].
PyAEDT INFO: PyAEDT version 1.9.0.
PyAEDT INFO: Initializing Desktop session.
PyAEDT INFO: AEDT version 2026.1.
PyAEDT INFO: Port 53725 session has been found.
PyAEDT INFO: AEDT installation Path C:\Program Files\ANSYS Inc\v261\AnsysEM
PyAEDT INFO: Connected to AEDT gRPC session on port 53725.
PyAEDT INFO: Non-graphical mode detected. Disabling Desktop logs.
```

```
DEBUG - Toolkit is connected to AEDT.
```

```
PyAEDT INFO: Python version 3.14.5 (tags/v3.14.5:5607950, May 10 2026, 10:43:50) [MSC v.
↪1944 64 bit (AMD64)].
PyAEDT INFO: PyAEDT version 1.9.0.
PyAEDT INFO: Returning found Desktop session with PID 328!
PyAEDT INFO: Project antenna_toolkit set to active.
PyAEDT INFO: Active Design set to HFSS_CWDT2T
PyAEDT INFO: AEDT objects correctly read
```

```
INFO - Toolkit is connected to AEDT design.
```

```
PyAEDT INFO: Desktop has been released.
```

```
INFO - AEDT is released.
```

```
[30]: True
```

4.2.22 Modify number of cores

```
[31]: properties.antenna.setup.num_cores = num_cores
```

4.2.23 Analyze design in batch mode

[32]:
```
toolkit_api.analyze()
```

```
DEBUG - Toolkit is not connected to AEDT.
DEBUG - Connecting AEDT.
```

```
PyAEDT INFO: Python version 3.14.5 (tags/v3.14.5:5607950, May 10 2026, 10:43:50) [MSC v.
↪1944 64 bit (AMD64)].
PyAEDT INFO: PyAEDT version 1.9.0.
PyAEDT INFO: Initializing Desktop session.
PyAEDT INFO: AEDT version 2026.1.
PyAEDT INFO: Port 53725 session has been found.
PyAEDT INFO: AEDT installation Path C:\Program Files\ANSYS Inc\v261\AnsysEM
PyAEDT INFO: Connected to AEDT gRPC session on port 53725.
PyAEDT INFO: Non-graphical mode detected. Disabling Desktop logs.
```

```
DEBUG - Toolkit is connected to AEDT.
```

```
PyAEDT INFO: Python version 3.14.5 (tags/v3.14.5:5607950, May 10 2026, 10:43:50) [MSC v.
↪1944 64 bit (AMD64)].
PyAEDT INFO: PyAEDT version 1.9.0.
PyAEDT INFO: Returning found Desktop session with PID 328!
PyAEDT INFO: Project antenna_toolkit set to active.
PyAEDT INFO: Active Design set to HFSS_CWDT2T
PyAEDT INFO: AEDT objects correctly read
```

```
INFO - Toolkit is connected to AEDT design.
```

```
PyAEDT INFO: Project antenna_toolkit Saved correctly
PyAEDT INFO: Project antenna_toolkit Saved correctly
PyAEDT INFO: Key Desktop/ActiveDSOConfigurations/HFSS correctly changed.
PyAEDT INFO: Solving all design setups. Analysis started...
PyAEDT INFO: Design setup None solved correctly in 0.0h 1.0m 21.0s
PyAEDT INFO: Key Desktop/ActiveDSOConfigurations/HFSS correctly changed.
PyAEDT INFO: Desktop has been released.
```

```
INFO - AEDT is released.
```

[32]:
```
True
```

4.2.24 Get scattering results

[33]:
```
scattering_data = toolkit_api.scattering_results()
```

```
DEBUG - Toolkit is not connected to AEDT.
DEBUG - Connecting AEDT.
```

```
PyAEDT INFO: Python version 3.14.5 (tags/v3.14.5:5607950, May 10 2026, 10:43:50) [MSC v.
↪1944 64 bit (AMD64)].
PyAEDT INFO: PyAEDT version 1.9.0.
PyAEDT INFO: Initializing Desktop session.
PyAEDT INFO: AEDT version 2026.1.
PyAEDT INFO: Port 53725 session has been found.
PyAEDT INFO: AEDT installation Path C:\Program Files\ANSYS Inc\v261\AnsysEM
PyAEDT INFO: Connected to AEDT gRPC session on port 53725.
PyAEDT INFO: Non-graphical mode detected. Disabling Desktop logs.
```

```
DEBUG - Toolkit is connected to AEDT.
```

```
PyAEDT INFO: Python version 3.14.5 (tags/v3.14.5:5607950, May 10 2026, 10:43:50) [MSC v.
↪1944 64 bit (AMD64)].
PyAEDT INFO: PyAEDT version 1.9.0.
PyAEDT INFO: Returning found Desktop session with PID 328!
PyAEDT INFO: Project antenna_toolkit set to active.
PyAEDT INFO: Active Design set to HFSS_CWDT2T
PyAEDT INFO: AEDT objects correctly read
```

```
INFO - Toolkit is connected to AEDT design.
```

```
PyAEDT INFO: Parsing C:\Users\ansys\AppData\Local\Temp\tmp6v9t96dr_ansys\pyaedt_prj_FK2\
↪antenna_toolkit.aedt.
PyAEDT INFO: File C:\Users\ansys\AppData\Local\Temp\tmp6v9t96dr_ansys\pyaedt_prj_FK2\
↪antenna_toolkit.aedt correctly loaded. Elapsed time: 0m 0sec
PyAEDT INFO: aedt file load time 0.03996682167053223
PyAEDT INFO: PostProcessor class has been initialized! Elapsed time: 0m 0sec
PyAEDT INFO: PostProcessor class has been initialized! Elapsed time: 0m 0sec
PyAEDT INFO: Post class has been initialized! Elapsed time: 0m 0sec
PyAEDT INFO: Modeler class has been initialized! Elapsed time: 0m 0sec
PyAEDT WARNING: No report category provided. Automatically identified Terminal Solution␣
↪Data
PyAEDT INFO: Solution Correctly loaded. Elapsed time: 0m 0sec
PyAEDT INFO: Solution Correctly parsed. Elapsed time: 0m 0sec
PyAEDT INFO: Desktop has been released.
```

```
INFO - AEDT is released.
```

4.2.25 Get farfield results

[34]:
```
frequency_str = [str(properties.antenna.synthesis.frequency) + properties.antenna.
↪synthesis.frequency_unit]
farfield_metadata, farfield_frequency = toolkit_api.export_farfield(
    frequencies=frequency_str, sphere="3D", encode=False
)
```

```
DEBUG - Toolkit is not connected to AEDT.
DEBUG - Connecting AEDT.
```

```
PyAEDT INFO: Python version 3.14.5 (tags/v3.14.5:5607950, May 10 2026, 10:43:50) [MSC v.
↪1944 64 bit (AMD64)].
PyAEDT INFO: PyAEDT version 1.9.0.
PyAEDT INFO: Initializing Desktop session.
PyAEDT INFO: AEDT version 2026.1.
PyAEDT INFO: Port 53725 session has been found.
PyAEDT INFO: AEDT installation Path C:\Program Files\ANSYS Inc\v261\AnsysEM
PyAEDT INFO: Connected to AEDT gRPC session on port 53725.
PyAEDT INFO: Non-graphical mode detected. Disabling Desktop logs.
```

```
DEBUG - Toolkit is connected to AEDT.
```

```
PyAEDT INFO: Python version 3.14.5 (tags/v3.14.5:5607950, May 10 2026, 10:43:50) [MSC v.
↪1944 64 bit (AMD64)].
PyAEDT INFO: PyAEDT version 1.9.0.
PyAEDT INFO: Returning found Desktop session with PID 328!
```

(continues on next page)

(continued from previous page)

```
PyAEDT INFO: Project antenna_toolkit set to active.
PyAEDT INFO: Active Design set to HFSS_CWDT2T
PyAEDT INFO: AEDT objects correctly read
```

```
INFO - Toolkit is connected to AEDT design.
```

```
PyAEDT INFO: Project antenna_toolkit Saved correctly
PyAEDT INFO: Far field sphere 3D is assigned
PyAEDT INFO: Exporting antenna metadata...
PyAEDT INFO: Parsing C:\Users\ansys\AppData\Local\Temp\tmp6v9t96dr_ansys\pyaedt_prj_FK2\
↪antenna_toolkit.aedt.
PyAEDT INFO: File C:\Users\ansys\AppData\Local\Temp\tmp6v9t96dr_ansys\pyaedt_prj_FK2\
↪antenna_toolkit.aedt correctly loaded. Elapsed time: 0m 0sec
PyAEDT INFO: aedt file load time 0.04127955436706543
PyAEDT INFO: PostProcessor class has been initialized! Elapsed time: 0m 0sec
PyAEDT INFO: PostProcessor class has been initialized! Elapsed time: 0m 0sec
PyAEDT INFO: Post class has been initialized! Elapsed time: 0m 0sec
PyAEDT INFO: Antenna metadata exported.
PyAEDT INFO: Exporting geometry...
PyAEDT INFO: Modeler class has been initialized! Elapsed time: 0m 0sec
PyAEDT INFO: Exporting embedded element patterns... Done: 1.2515673637390137 seconds
PyAEDT INFO: Desktop has been released.
```

```
INFO - AEDT is released.
```

4.2.26 Get antenna model

[35]:
```
files = toolkit_api.export_aedt_model(encode=False)
```

```
DEBUG - Toolkit is not connected to AEDT.
DEBUG - Connecting AEDT.
```

```
PyAEDT INFO: Python version 3.14.5 (tags/v3.14.5:5607950, May 10 2026, 10:43:50) [MSC v.
↪1944 64 bit (AMD64)].
PyAEDT INFO: PyAEDT version 1.9.0.
PyAEDT INFO: Initializing Desktop session.
PyAEDT INFO: AEDT version 2026.1.
PyAEDT INFO: Port 53725 session has been found.
PyAEDT INFO: AEDT installation Path C:\Program Files\ANSYS Inc\v261\AnsysEM
PyAEDT INFO: Connected to AEDT gRPC session on port 53725.
PyAEDT INFO: Non-graphical mode detected. Disabling Desktop logs.
```

```
DEBUG - Toolkit is connected to AEDT.
```

```
PyAEDT INFO: Python version 3.14.5 (tags/v3.14.5:5607950, May 10 2026, 10:43:50) [MSC v.
↪1944 64 bit (AMD64)].
PyAEDT INFO: PyAEDT version 1.9.0.
PyAEDT INFO: Returning found Desktop session with PID 328!
PyAEDT INFO: Project antenna_toolkit set to active.
PyAEDT INFO: Active Design set to HFSS_CWDT2T
PyAEDT INFO: AEDT objects correctly read
```

```
INFO - Toolkit is connected to AEDT design.
```

```
PyAEDT INFO: Project antenna_toolkit Saved correctly
PyAEDT INFO: Parsing C:\Users\ansys\AppData\Local\Temp\tmp6v9t96dr_ansys\pyaedt_prj_FK2\
```

(continues on next page)

(continued from previous page)

```
↪antenna_toolkit.aedt.
PyAEDT INFO: File C:\Users\ansys\AppData\Local\Temp\tmp6v9t96dr_ansys\pyaedt_prj_FK2\
↪antenna_toolkit.aedt correctly loaded. Elapsed time: 0m 0sec
PyAEDT INFO: aedt file load time 0.044783592224121094
PyAEDT INFO: PostProcessor class has been initialized! Elapsed time: 0m 0sec
PyAEDT INFO: PostProcessor class has been initialized! Elapsed time: 0m 0sec
PyAEDT INFO: Post class has been initialized! Elapsed time: 0m 0sec
PyAEDT INFO: Modeler class has been initialized! Elapsed time: 0m 0sec
PyAEDT INFO: Desktop has been released.
```

```
INFO - AEDT is released.
```

4.2.27 Release AEDT

Release AEDT.

[36]:
```
toolkit_api.release_aedt(True, True)
```

```
DEBUG - Toolkit is not connected to AEDT.
DEBUG - Connecting AEDT.
```

```
PyAEDT INFO: Python version 3.14.5 (tags/v3.14.5:5607950, May 10 2026, 10:43:50) [MSC v.
↪1944 64 bit (AMD64)].
PyAEDT INFO: PyAEDT version 1.9.0.
PyAEDT INFO: Initializing Desktop session.
PyAEDT INFO: AEDT version 2026.1.
PyAEDT INFO: Port 53725 session has been found.
PyAEDT INFO: AEDT installation Path C:\Program Files\ANSYS Inc\v261\AnsysEM
PyAEDT INFO: Connected to AEDT gRPC session on port 53725.
PyAEDT INFO: Non-graphical mode detected. Disabling Desktop logs.
```

```
DEBUG - Toolkit is connected to AEDT.
```

```
PyAEDT INFO: Desktop has been released and closed.
```

```
INFO - AEDT is released.
```

[36]:
```
True
```

4.2.28 Plot results

Plot exported 3D model

[37]:
```
model_plotter = ModelPlotter()
for file in files:
    model_plotter.add_object(file[0], file[1], file[2])
```

[38]:
```
# Plot the model
model_plotter.plot(show=True)
```

```
C:\actions-runner\_work\pyaedt-toolkits-antenna\pyaedt-toolkits-antenna\.venv\Lib\site-
↪packages\ansys\aedt\core\visualization\plot\pyvista.py:1366: PyVistaDeprecationWarning:
↪ ``Plotter.button_widgets`` is deprecated; use ``Plotter.widgets.button_widgets``␣
↪instead.
  while len(self.plot.button_widgets) > 1:
```

```
EmbeddableWidget(value='<iframe srcdoc="<!doctype html>\n<html lang=&quot;en&quot;>\n
↪<head>\n    <meta chars
```

[38]:
```
True
```

4.2.29 Load far field

[39]:
```
farfield_data = FfdSolutionData(farfield_metadata)
```

4.2.30 Plot far field 3D

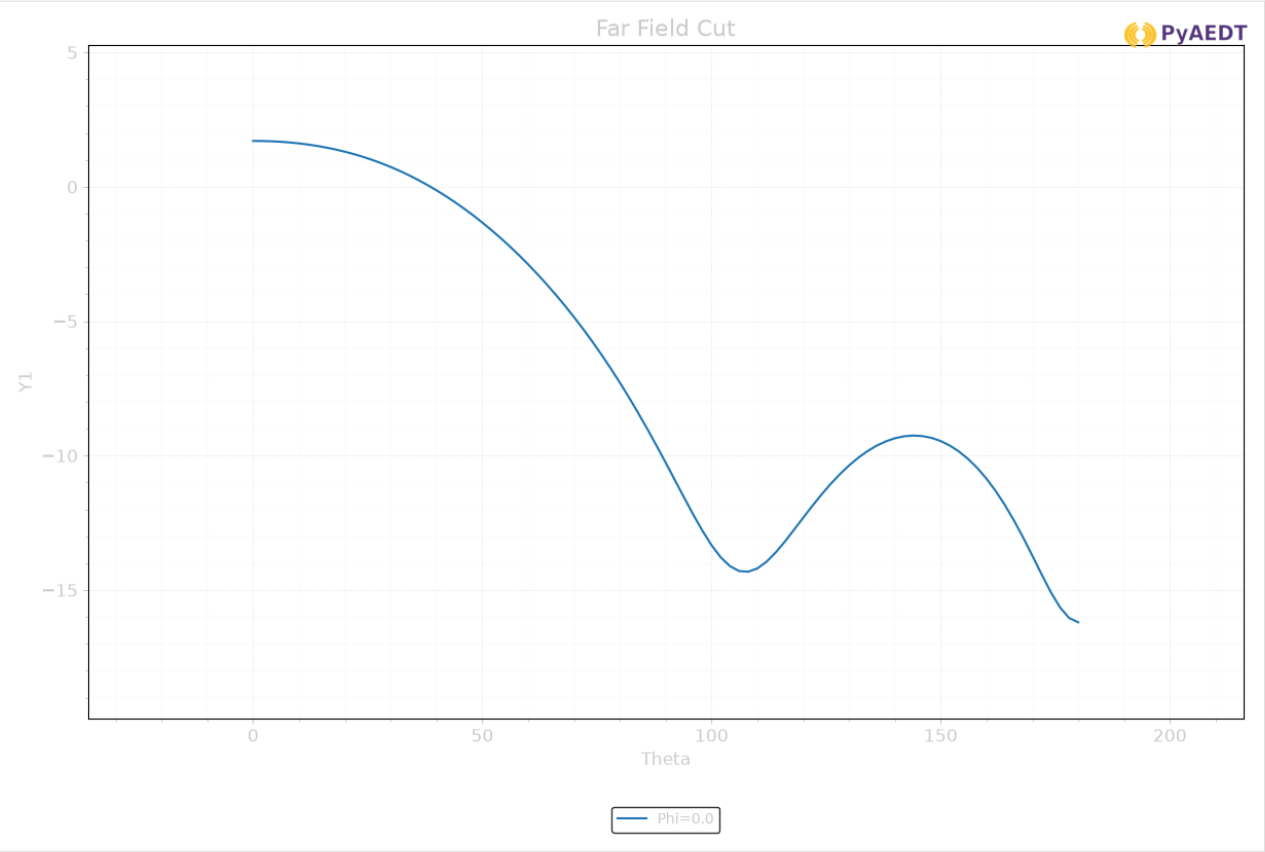
[40]:
```
# Render 3D far field - in Jupyter, this will show as interactive 3D
pyvista_plotter = pv.Plotter(notebook=True, off_screen=non_graphical)
farfield_data.plot_3d(
    quantity="RealizedGain",
    show=False,
    pyvista_object=pyvista_plotter,
)
pyvista_plotter.show(jupyter_backend="html")
```

```
EmbeddableWidget(value='<iframe srcdoc="<!doctype html>\n<html lang=&quot;en&quot;>\n
↪<head>\n    <meta chars
```

4.2.31 Plot far field cut

[41]:
```
# Render far field cut plot
farfield_data.plot_cut(primary_sweep="theta", show=True)
```

[41]:
```
Class: ansys.aedt.core.visualization.plot.matplotlib.ReportPlotter
```



4.2.32 Clean temporary directory

[42]:
```
temp_dir.cleanup()
```

CHAPTER

FIVE

CONTRIBUTE

Overall guidance on contributing to a PyAnsys repository appears in Contributing in the *PyAnsys Developers Guide*. Ensure that you are thoroughly familiar with this guide before attempting to contribute to PyAEDT or its toolkits.

The following contribution information is specific to PyAEDT toolkits.

5.1 Clone the repository

To clone and install the latest version of this toolkit in development mode, run:

```
git clone https://github.com/ansys/pyaedt-toolkits-antenna.git
cd pyaedt-toolkits-antenna
python -m pip install --upgrade pip
pip install -e .
```

5.2 Add new antennas

TBD

5.3 Post issues

Use the AEDT Antenna Toolkit Issues page to report bugs and request new features.

5.4 View documentation

Documentation for the latest stable release is hosted at https://aedt.antenna.toolkit.docs.pyansys.com/.

In the upper right corner of the documentations title bar, there is an option for switching from viewing the documentation for the latest stable release to viewing the documentation for the development version or previously released versions.

5.5 Adhere to code style

PyAEDT toolkit is compliant with PyAnsys code style. It uses the tool pre-commit to select the code style. You can install and activate this tool with:

```
pip install pre-commit
pre-commit run --all-files
```

You can also install this as a pre-commit hook with:

```
pre-commit install
```

This way, its not possible for you to push code that fails the style checks. For example:

```
$ pre-commit install
$ git commit -am "Add my cool feature."
black....................................................................Passed
isort (python)...........................................................Passed
flake8...................................................................Passed
codespell................................................................Passed
fix requirements.txt.....................................................Passed
blacken-docs.............................................................Passed
```

5.5.1 Maximum line length

Best practice is to keep the length at or below 120 characters for code and comments. Lines longer than this might not display properly on some terminals and tools or might be difficult to follow.

CHAPTER

SIX

RELEASE NOTES

This document contains the release notes for the project.

6.1 0.9.1 - June 25, 2026

Added

CLI	#408

Dependencies

Bump actions/checkout from 6.0.2 to 6.0.3	#412
Bump astral-sh/setup-uv from 8.1.0 to 8.2.0	#413
Bump prek from 0.4.3 to 0.4.4	#414
Bump ruff-pre-commit from v0.15.15 to 0.15.16	#415
Bump check-jsonschema from 0.37.2 to 0.37.3	#416
Bump codecov/codecov-action from 6.0.1 to 7.0.0	#417
Bump ansys-sphinx-theme from 1.8.2 to 1.9.0	#418

Maintenance

Update CHANGELOG for v0.9.0	#406
Bump dev version into v0.10.dev0	#407
Add typer	#409
Update missing or outdated files	#419
Add Synopsys word	#421

6.2 0.9.0 - June 08, 2026

Added

Update run toolkit	#339
Add ACT custom GPS patch family	#367
Add dipole antenna family	#368
Add helix antenna family models	#369
Add misc antenna family	#370
Add monopole antenna family	#371
Add grouped patch family models	#372
Add PIFA antenna family	#373
Add planar spiral family support	#374
Add slot antenna family	#376
Add Vivaldi antenna family	#377
Add waveguide antenna family	#378
Add YagiUda antenna family	#379
Initial implementation log periodic	#398
Add reference view and remove Reflector section	#400

Fixed

Remove unwanted changelog related to closed PR 360	#389

Documentation

Update installation.rst	#335
Fixed getting started link	#341
Update `contributors.md` with the latest contributors	#342
Fix vtk-osmesa install step	#351
Update ``CONTRIBUTORS.md`` with the latest contributors	#357
Added docstrings and author notes	#397
Update bad workflow in README.rst	#404

Dependencies

Bump ansys/actions from 9 to 10	#334
Bump ansys/actions from 10.2.12 to 10.3.0	#388
Bump actions/labeler from 6.0.1 to 6.1.0	#390
Bump ansys/actions from 10.3.0 to 10.3.2	#392
Bump codecov/codecov-action from 6.0.0 to 6.0.1	#395
Bump ruff-pre-commit from v0.15.14 to 0.15.15	#403

Maintenance

update CHANGELOG for v0.8.0	#331
Update dev version	#332
Fix release job	#333
Pin versions of actions in GitHub workflow files with SHA	#355
Add ``ansys/actions/check-actions-security`` action and related fixes (updated)	#384
Update missing or outdated files	#386, #396
Add dependabot cooldown settings	#387
General updates (CI, config files, dependencies)	#399

6.3 0.8.0 - May 20, 2025

Dependencies

Bump actions/download-artifact from 4.1.9 to 4.3.0	#330

Maintenance

Update v0.8.dev0	#320
update CHANGELOG for v0.7.1	#325
Fix codecov job	#327
Update CICD to Python 3.12	#328

6.4 0.7.1 - May 08, 2025

Fixed

Upload artifacts	#318
Fix release artifacts	#323

Maintenance

update CHANGELOG for v0.6.0	#317
Fix release process	#321

6.5 0.6.0 - May 06, 2025

Dependencies

Bump codecov/codecov-action from 4 to 5	#315

Maintenance

Add Changelog #314

CHAPTER

SEVEN

INDICES AND TABLES

- genindex
- search

BIBLIOGRAPHY

[1] C. Balanis, Wideband and Travelling-Wave Antennas, *Modern Antenna Handbook*, New York, 2008.

[1] C. Balanis, Wideband and Travelling-Wave Antennas, *Modern Antenna Handbook*, New York, 2008.

[1] C. Balanis, Wideband and Travelling-Wave Antennas, *Modern Antenna Handbook*, New York, 2008.

[1] R. Johnson, Frequency Independent Antennas, Antenna Engineering Handbook, 3rd ed. New York, McGraw-Hill, 1993.

[1] C. Balanis, Wideband and Travelling-Wave Antennas, *Modern Antenna Handbook*, New York, 2008.

[1] C. Balanis, Wideband and Travelling-Wave Antennas, *Modern Antenna Handbook*, New York, 2008.

[1] C. Balanis, Wideband and Travelling-Wave Antennas, *Modern Antenna Handbook*, New York, 2008.

[1] C. Balanis, Wideband and Travelling-Wave Antennas, *Modern Antenna Handbook*, New York, 2008.

[1] C. Balanis, Wideband and Travelling-Wave Antennas, *Modern Antenna Handbook*, New York, 2008.

[1] C. Balanis, Aperture Antennas: Analysis, Design, and Applications, *Modern Antenna Handbook*, New York, 2008.

[1] C. Balanis, Horn Antennas, Antenna Theory Analysis, 3rd ed. Hoboken: Wiley, 2005, sec. 13.6, pp. 785-791.

[1] C. Balanis, Aperture Antennas: Analysis, Design, and Applications, *Modern Antenna Handbook*, New York, 2008.

[1] C. Balanis, Aperture Antennas: Analysis, Design, and Applications, *Modern Antenna Handbook*, New York, 2008.

[1] C. Balanis, Aperture Antennas: Analysis, Design, and Applications, *Modern Antenna Handbook*, New York, 2008.

[1] C. Balanis, Aperture Antennas: Analysis, Design, and Applications, *Modern Antenna Handbook*, New York, 2008.

[1] C. Balanis, Aperture Antennas: Analysis, Design, and Applications, *Modern Antenna Handbook*, New York, 2008.

[1] K. L. Walton and V. C. Sundberg, Broadband ridged horn design, Microwave J., vol. 4, pp. 96-101, Apr. 1964.

[2] C. Bruns et al., Analysis and Simulations of a 1-18 GHz Broadband Double-Ridged Horn Antenna, IEEE Electromag. Compat., vol. 45, pp. 55-60, Feb 2003.

[3] C. Balanis, Horn Antennas, Antenna Theory Analysis, 3rd ed. Hoboken: Wiley, 2005, ch. 13.

[1] C. Balanis, Microstrip Antennas, *Antenna Theory*, 2nd Ed. New York: Wiley, 1997.

[1] C. Balanis, Microstrip Antennas, *Antenna Theory*, 2nd Ed. New York: Wiley, 1997.

[1] C. Balanis, Microstrip Antennas, *Antenna Theory*, 2nd Ed. New York: Wiley, 1997.